

LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE



Oleh:

Muhammad Rizky

NIM. 2310817310011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rizky
NIM : 2310817310011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	5
DAFTAR TABEL	6
MODUL 1 : Variabel, Operator, dan Array	7
SOAL 1	7
A. Source Code.....	9
B. Output Program	12
C. Pembahasan	13
MODUL 2 : Android Layout.....	17
SOAL 1	17
A. Source Code.....	18
B. Output Program	23
C. Pembahasan	24
MODUL 3 : Build a Scrollable List	31
SOAL 1	31
A. Source Code.....	31
B. Output Program	40
C. Pembahasan	42
SOAL 2.....	49
A. Jawaban	49
MODUL 4 : ViewModel and Debugging.....	50
SOAL 1	50

A. Source Code.....	50
B. Output Program	61
C. Pembahasan	62
SOAL 2.....	72
A. Jawaban	72
MODUL 5 : Connect to the Internet.....	73
SOAL 1	73
A. Source Code.....	73
B. Output Program	88
C. Pembahasan	90
TAUTAN GIT	99

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1	12
Gambar 2. Screenshot Hasil Jawaban Soal 1	13
Gambar 3. Screenshot Hasil Jawaban Soal 1	13
Gambar 4. Screenshot Splashscreen Aplikasi	23
Gambar 5. Screenshot Tampilan Aplikasi.....	23
Gambar 6. Screenshot Tampilan Aplikasi.....	24
Gambar 7. Screenshot Tampilan Aplikasi.....	24
Gambar 8. Screenshot Halaman Awal.....	40
Gambar 9. Screenshot Ketika Tombol BWF Ditekan Mengarah Ke Website	41
Gambar 10. Screenshot Ketika Tombol Detail Ditekan	41
Gambar 11. Screenshot Halaman Awal.....	61
Gambar 12. Screenshot Ketika Tombol Ditekan Mengarah Ke Website	61
Gambar 13. Screenshot Ketika Tombol Detail Ditekan	62
Gambar 14. Screenshot Halaman Awal.....	88
Gambar 15. Screenshot Ketika Tombol Ditekan Mengarah Ke Aplikasi Youtube.....	88
Gambar 16. Screenshot Ketika Tombol Favorite Ditekan	89
Gambar 17. Screenshot Halaman Favorite	89
Gambar 18. Screenshot Ketika Tombol Hapus Dari Favorite Ditekan	90

DAFTAR TABEL

Table 1. Source Code Jawaban Soal 1	9
Table 2. Source Code Jawaban Soal 1	11
Table 3. Source Code Jawaban Soal 1	18
Table 4. Source Code Jawaban Soal 1	21
Table 5. Source Code MainActivity.kt	31
Table 6. Source Code DataTournament.kt	32
Table 7. Source Code NaavGraph.kt	34
Table 8. Souce Code ListScreen.kt	35
Table 9. Source Code DetailScreen.kt	37
Table 10. Source Code MainActivity.kt	50
Table 11. Source Code DataTournament.kt	51
Table 12. Source Code Item.kt	53
Table 13. Source Code NavGraph.kt	53
Table 14. Source Code DetailScreen.kt	54
Table 15. Source Code ListScreen.kt	56
Table 16. Source Code ItemViewModel.kt	58
Table 17. Source Code ItemViewModelFactory.kt	59
Table 18. Source Code MainActivity.kt	60
Table 19. Source Code MainActivity.kt	73
Table 20. Source Code ApiService.kt	81
Table 21. Source Code RetrofitInstance.kt	81
Table 22. Source Code PostDao.kt	82
Table 23. Source Code PostDatabase.kt	82
Table 24. Source Code Post.kt	83
Table 25. Source Code PostRepository.kt	83
Table 26. Source Code DetailScreen.kt	84
Table 27. Source Code PostViewModel.kt	85

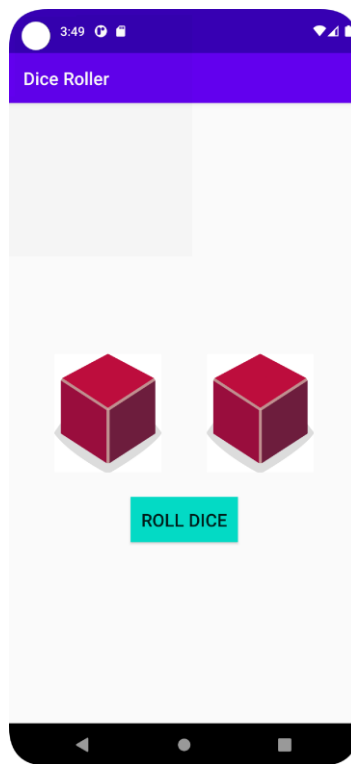
MODUL 1 : Variabel, Operator, dan Array

SOAL 1

Soal Praktikum:

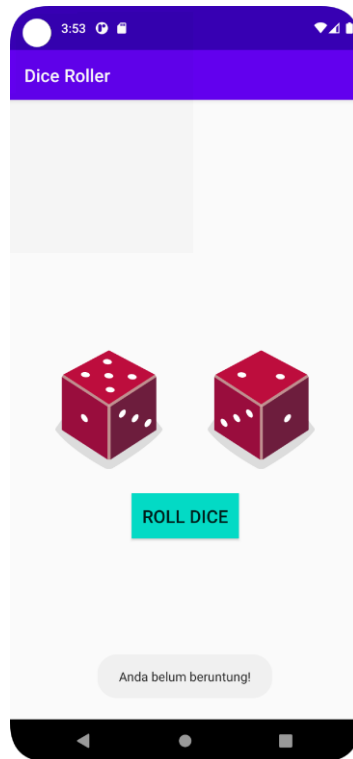
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



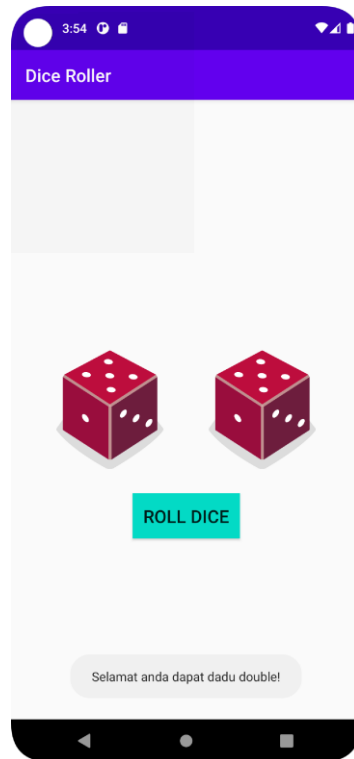
Gambar 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2 Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.
4. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.
5. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2IIH5qin3z5ta7H9y2N_5OMW81Ll&export=download



Gambar 3 Tampilan Roll Dadu Double

A. Source Code

MainActivity.kt

Table 1. Source Code Jawaban Soal 1

1	package com.example.prak_01
2	
3	import android.os.Bundle
4	import android.widget.Toast
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.activity.enableEdgeToEdge
8	import androidx.compose.foundation.Image
9	import androidx.compose.foundation.layout.Arrangement
10	import androidx.compose.foundation.layout.Column
11	import androidx.compose.foundation.layout.Row
12	import androidx.compose.foundation.layout.fillMaxSize
13	import androidx.compose.material3.Button
14	import androidx.compose.material3.ExperimentalMaterial3Api
15	import androidx.compose.material3.Text
16	import androidx.compose.material3.TopAppBar
17	import androidx.compose.material3.TopAppBarDefaults
18	import androidx.compose.runtime.Composable

```

19 import androidx.compose.runtime.getValue
20 import androidx.compose.ui.Alignment
21 import androidx.compose.ui.Modifier
22 import androidx.compose.ui.draw.scale
23 import androidx.compose.ui.graphics.Color
24 import androidx.compose.ui.platform.LocalContext
25 import androidx.compose.ui.res.painterResource
26 import androidx.compose.ui.tooling.preview.Preview
27 import androidx.lifecycle.viewmodel.compose.viewModel
28 import com.example.prak_01.ui.theme.Prak01Theme
29
30 class MainActivity : ComponentActivity() {
31     override fun onCreate(savedInstanceState: Bundle?) {
32         super.onCreate(savedInstanceState)
33         enableEdgeToEdge()
34         setContent {
35             Prak01Theme {
36                 Dadu()
37             }
38         }
39     }
40 }
41
42 @OptIn(ExperimentalMaterial3Api::class)
43 @Composable
44 fun Dadu(viewModel: DaduViewModel = viewModel()) {
45     val nilaiDadu1 by viewModel.nilaiDadu1
46     val nilaiDadu2 by viewModel.nilaiDadu2
47
48     TopAppBar(
49         title = {
50             Text(
51                 text = "Dice Roller",
52                 color = Color.White
53             )
54         },
55         colors = TopAppBarDefaults.topAppBarColors(
56             containerColor = Color.Magenta
57         )
58     )
59
60     Column(
61         modifier = Modifier
62             .fillMaxSize()
63             .scale(0.8f),
64         horizontalAlignment = Alignment.CenterHorizontally,
65         verticalArrangement = Arrangement.Center) {
66         Row {
67             Image(
68                 painter = painterResource(acak(nilaiDadu1)),
69                 contentDescription = "dicel",
70             )

```

71	Image(
72	painter = painterResource(acak(nilaiDadu2)),
73	contentDescription = "dice2",
74	)
75	}
76	Button(
77	modifier = Modifier
78	.scale(1.5f),
79	onClick = {
80	viewModel.rollDice()
81	}) {
82	Text(
83	text = "Roll Dice",
84	)
85	}
86	}
87	
88	val context = LocalContext.current
89	if(nilaiDadu1 == nilaiDadu2){
90	Toast.makeText(context, "Selamat anda dapat dadu
91	double!", Toast.LENGTH_SHORT).show()
92	}
93	if(nilaiDadu1 != nilaiDadu2){
94	Toast.makeText(context, "Anda belum beruntung!",
95	Toast.LENGTH_SHORT).show()
96	}
97	}
98	
99	fun acak(nilaiDadu: Int): Int {
100	return when (nilaiDadu){
101	1 -> R.drawable.dice_1
102	2 -> R.drawable.dice_2
103	3 -> R.drawable.dice_3
104	4 -> R.drawable.dice_4
105	5 -> R.drawable.dice_5
106	else -> R.drawable.dice_6
107	}
108	}
109	
110	@Preview(showBackground = true)
111	@Composable
112	fun DaduPreview() {
113	Dadu()
114	}

DaduViewModel.kt

Table 2. Source Code Jawaban Soal 1

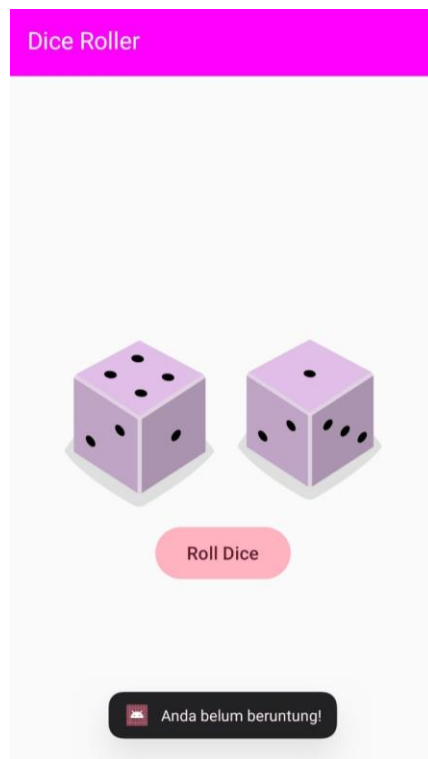
1	package com.example.prak_01
2	

```

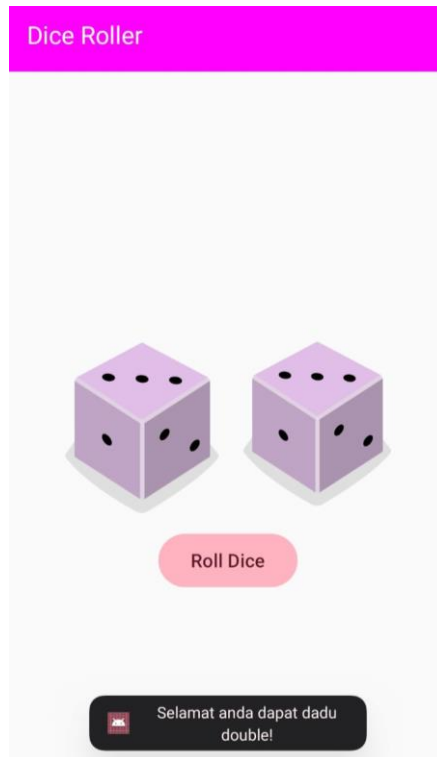
3 import androidx.compose.runtime.State
4 import androidx.compose.runtime.mutableIntStateOf
5 import androidx.lifecycle.ViewModel
6 import kotlin.random.Random
7
8 class DaduViewModel : ViewModel(){
9     private val _nilaiDadu1 = mutableIntStateOf(1)
10    val nilaiDadu1: State<Int> = _nilaiDadu1
11
12    private val _nilaiDadu2 = mutableIntStateOf(1)
13    val nilaiDadu2: State<Int> = _nilaiDadu2
14
15    fun rollDice(){
16        _nilaiDadu1.intValue = Random.nextInt(1,7)
17        _nilaiDadu2.intValue = Random.nextInt(1,7)
18    }
19 }

```

B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1



Gambar 2. Screenshot Hasil Jawaban Soal 1



Gambar 3. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

MainActivity.kt:

Line 1, Dideklarasikan nama package file Kotlin, yaitu com.example.prak_01

Line 3, Diimpor Bundle dari Android, yang digunakan untuk menyimpan dan mengambil data antar aktivitas

Line 4, Diimpor Toast, yang digunakan untuk menampilkan pesan pop up di layar

Line 5, Diimpor ComponentActivity, kelas dasar untuk Jetpack Compose

Line 6, Diimpor setContent, fungsi untuk menentukan isi tampilan menggunakan Jetpack Compose

Line 7, Diimpor enableEdgeToEdge, digunakan untuk mengaktifkan tampilan full screen yang mencakup area status bar dan navigation bar

Line 8 - 9, Diimpor elemen-elemen UI Compose seperti, Image, Column, dan Row untuk mengatur layout aplikasi

Line 10, Diimpor fillMaxSize, modifier untuk membuat elemen mengisi seluruh ukuran layar

Line 11, Diimpor Button, komponen tombol dari Material 3

Line 12, Diimpor ExperimentalMaterial3Api, untuk menandai bahwa TopAppBar masih eksperimental

Line 13 - 14, Diimpor Text dan TopAppBar, komponen UI untuk teks dan AppBar atas dari Material 3

Line 15, Diimpor TopAppBarDefaults, digunakan untuk mengatur warna default dari AppBar

Line 16 - 17, Diimpor Composable dan getValue, yang digunakan dalam fungsi-fungsi Compose dan delegasi nilai state

Line 18, Diimpor Alignment untuk mengatur perataan elemen dalam layout Compose

Line 19, Diimpor Modifier untuk memodifikasi elemen UI seperti ukuran dan posisi

Line 20, Diimpor scale, modifier untuk mengatur skala ukuran elemen

Line 21, Diimpor Color untuk mengatur warna elemen UI seperti teks dan latar belakang

Line 22, Diimpor LocalContext, digunakan untuk mengakses konteks lokal dari Compose

Line 23, Diimpor painterResource, digunakan untuk menampilkan gambar dari resource drawable

Line 24, Diimpor tooling.preview, digunakan untuk menampilkan preview UI Compose

Line 25, Diimpor viewModel dari lifecycle untuk mengakses ViewModel dalam Compose

Line 26, Diimpor Prak01Theme dari folder tema untuk menerapkan tema ke aplikasi

Line 28, Didefinisikan kelas MainActivity yang mewarisi ComponentActivity

Line 29 - 36, onCreate dijalankan saat aktivitas dibuat Fungsi enableEdgeToEdge() diaktifkan dan setContent dipanggil untuk menampilkan UI Dadu() dengan tema Prak01Theme

Line 38, @OptIn(ExperimentalMaterial3Api::class) digunakan karena TopAppBar masih bersifat eksperimental

Line 39, Dideklarasikan fungsi Dadu() sebagai fungsi @Composable Parameter viewModel menggunakan fungsi viewModel() untuk mendapatkan instans dari DaduViewModel

Line 40 - 41, Nilai dadu 1 dan dadu 2 diambil dari ViewModel menggunakan by

Line 43 - 50, TopAppBar ditampilkan dengan teks Dice Roller dan latar belakang berwarna magenta

Line 52 - 61, Layout utama menggunakan Column untuk menata elemen secara vertikal di tengah layar dan menggunakan scale(0.8f) untuk memperkecil ukuran elemen dalam kolom

Line 53 - 58, Row menampilkan dua gambar dadu secara horizontal menggunakan Image yang sumber gambarnya diatur berdasarkan fungsi acak(nilaiDadu)

Line 59 - 63, Tombol Roll Dice dengan scale(1.5f) yang akan memperbesar button dan akan memanggil fungsi rollDice() dari ViewModel saat ditekan

Line 65, Variabel context diisi dengan konteks lokal dari Compose digunakan untuk menampilkan Toast

Line 66 - 67, Jika nilai kedua dadu sama maka Toast akan muncul dengan pesan Selamat anda dapat dadu double!

Line 68 - 69, Jika nilai kedua dadu berbeda maka Toast akan muncul dengan pesan Anda belum beruntung!

Line 71 - 79, Fungsi acak(nilaiDadu: Int) digunakan untuk mengembalikan ID gambar drawable berdasarkan nilai dadu 1-6

Line 81 - 84, Fungsi DaduPreview() adalah fungsi preview yang menampilkan UI Dadu() di Android Studio Preview

DaduViewModel.kt

Line 1, Dideklarasikan nama package file Kotlin, yaitu `com.example.prak_01`

Line 3, Diimpor `State` dari `Compose`, yang digunakan untuk menyimpan dan membaca nilai state yang bersifat immutable sehingga tidak dapat diubah langsung dari luar

Line 4, Diimpor `mutableIntStateOf`, fungsi `Compose` untuk membuat state yang dapat diubah dengan tipe data `Int`

Line 5, Diimpor `ViewModel` dari `Android Lifecycle`, yang digunakan untuk mengelola data UI secara terpisah dari UI itu sendiri dan tetap bertahan saat konfigurasi berubah

Line 6, Diimpor `Random` dari Kotlin, digunakan untuk menghasilkan angka acak

Line 8, Dideklarasikan kelas `DaduViewModel` yang merupakan turunan dari `ViewModel`

Line 9, Dibuat variabel `_nilaiDadu1` bertipe `mutableIntStateOf(1)` yang menyimpan nilai acak untuk dadu pertama Variabel ini bersifat `private`

Line 10, Dibuat variabel `nilaiDadu1` bertipe `State<Int>`, yang merupakan versi publik dari `_nilaiDadu1` agar hanya bisa dibaca dari luar tanpa bisa diubah

Line 12, Dibuat variabel `_nilaiDadu2` bertipe `mutableIntStateOf(1)` yang menyimpan nilai acak untuk dadu pertama Variabel ini bersifat `private`

Line 13, Dibuat variabel `nilaiDadu2` bertipe `State<Int>`, yang merupakan versi publik dari `_nilaiDadu2` agar hanya bisa dibaca dari luar tanpa bisa diubah

Line 15, Didefinisikan fungsi `rollDice()` yang akan dijalankan ketika pengguna menekan tombol untuk melempar dadu

Line 16, Mengubah nilai `_nilaiDadu1` dengan angka acak dari 1 sampai 6 menggunakan `Random.nextInt(1, 7)`

Line 17, Mengubah nilai `_nilaiDadu2` dengan angka acak dari 1 sampai 6 menggunakan `Random.nextInt(1, 7)`

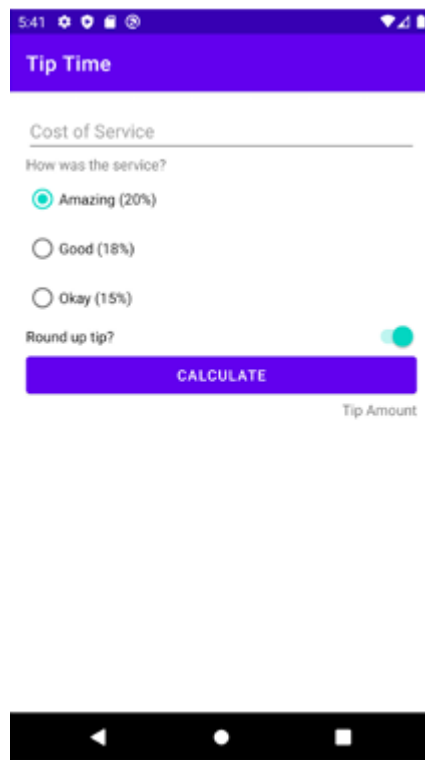
MODUL 2 : Android Layout

SOAL 1

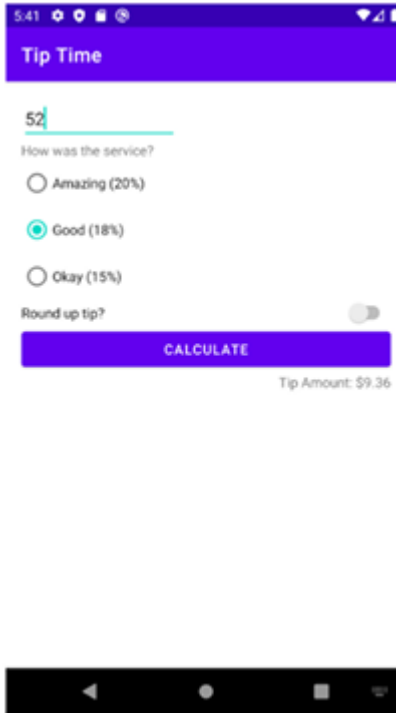
Soal Praktikum:

Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

1. Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
2. Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
3. Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
4. Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1 Tampilan Awal Aplikasi



Gambar 2 Tampilan Aplikasi Setelah Dijalankan

A. Source Code

MainActivity.kt

Table 3. Source Code Jawaban Soal 1

1	package com.example.modul_2
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.compose.foundation.layout.Arrangement
8	import androidx.compose.foundation.layout.Column
9	import androidx.compose.foundation.layout.Row
10	import androidx.compose.foundation.layout.Spacer
11	import androidx.compose.foundation.layout.fillMaxSize
12	import androidx.compose.foundation.layout.fillMaxWidth
13	import androidx.compose.foundation.layout.height
14	import androidx.compose.foundation.layout.padding
15	import androidx.compose.material3.Button
16	import androidx.compose.material3.ButtonDefaults
17	import androidx.compose.material3.ExperimentalMaterial3Api
18	import androidx.compose.material3.MaterialTheme
19	import androidx.compose.material3.OutlinedTextField
20	import androidx.compose.material3.RadioButton

```

21 import androidx.compose.material3.RadioButtonDefaults
22 import androidx.compose.material3.Scaffold
23 import androidx.compose.material3.Switch
24 import androidx.compose.material3.SwitchDefaults
25 import androidx.compose.material3.Text
26 import androidx.compose.material3.TopAppBar
27 import androidx.compose.material3.TopAppBarDefaults
28 import androidx.compose.runtime.Composable
29 import androidx.compose.runtime.collectAsState
30 import androidx.compose.runtime.getValue
31 import androidx.compose.ui.Alignment
32 import androidx.compose.ui.Modifier
33 import androidx.compose.ui.graphics.Color
34 import androidx.compose.ui.text.style.TextAlign
35 import androidx.compose.ui.tooling.preview.Preview
36 import androidx.compose.ui.unit.dp
37 import
38 androidx.core.splashscreen.SplashScreen.Companion.installSplashScreen
39 import androidx.lifecycle.viewmodel.compose.viewModel
40 import androidx.compose.foundation.rememberScrollState
41 import androidx.compose.foundation.verticalScroll
42 import com.example.modul_2.ui.theme.Modul_2Theme
43
44 class MainActivity : ComponentActivity() {
45     override fun onCreate(savedInstanceState: Bundle?) {
46         super.onCreate(savedInstanceState)
47         installSplashScreen()
48         enableEdgeToEdge()
49         setContent {
50             Modul_2Theme {
51                 TipTime()
52             }
53         }
54     }
55 }
56
57 @OptIn(ExperimentalMaterial3Api::class)
58 @Composable
59 fun TipTime(viewModel: TipTimeViewModel = viewModel()) {
60     val costInput by viewModel.costInput.collectAsState()
61     val selectedPercentage by
62     viewModel.selectedPercentage.collectAsState()
63     val roundTip by viewModel.roundTip.collectAsState()
64     val tipAmount by viewModel.tipAmount.collectAsState()
65
66     val radioOptions = listOf("Amazing (20%)" to 0.20, "Good (18%)" to
67     0.18, "Okay (15%)" to 0.15)
68
69     Scaffold(
70         topBar = {
71             TopAppBar(
72                 title = { Text("Tip Time", color = Color.White) },

```

```

73         colors
74 TopAppBarDefaults.topAppBarColors(containerColor = Color.Red)
75     )
76 }
77 ) { innerPadding ->
78     Column(
79         modifier = Modifier
80             .fillMaxSize()
81             .padding(innerPadding)
82             .padding(16.dp)
83             .verticalScroll(rememberScrollState())
84     ) {
85         Spacer(modifier = Modifier.height(8.dp))
86
87         OutlinedTextField(
88             value = costInput,
89             onValueChange = viewModel::onCostChanged,
90             label = { Text("Cost of service") },
91             singleLine = true,
92             modifier = Modifier.fillMaxWidth()
93         )
94
95         Text("How was the service?", style =
96 MaterialTheme.typography.titleMedium)
97         Spacer(modifier = Modifier.height(8.dp))
98
99         radioOptions.forEach { option ->
100             Row(verticalAlignment = Alignment.CenterVertically) {
101                 RadioButton(
102                     selected = selectedPercentage ==
103 option.second,
104                     onClick = {
105 viewModel.onPercentageChanged(option.second) },
106                     colors = RadioButtonDefaults.colors(
107                         selectedColor = Color.Yellow,
108                         unselectedColor = Color.Gray,
109                     )
110                 )
111                 Text(text = option.first, modifier =
112 Modifier.padding(start = 8.dp))
113             }
114         }
115
116         Row(
117             verticalAlignment = Alignment.CenterVertically,
118             horizontalArrangement = Arrangement.SpaceBetween,
119             modifier = Modifier.fillMaxWidth()
120         ) {
121             Text("Round tip?")
122             Switch(
123                 checked = roundTip,
124                 onCheckedChange = viewModel::onRoundTipChanged,

```

```

125         colors = SwitchDefaults.colors(
126             checkedThumbColor = Color.Red,
127             checkedTrackColor = Color.White,
128             checkedBorderColor = Color.Red
129         )
130     )
131 }
132
133 Button(
134     onClick = viewModel::calculateTip,
135     colors = ButtonDefaults.buttonColors(containerColor =
136 Color.Red),
137     modifier = Modifier.fillMaxWidth()
138 ) {
139     Text("Calculate")
140 }
141
142 Text(
143     text = tipAmount,
144     modifier = Modifier.fillMaxWidth(),
145     textAlign = TextAlign.End
146 )
147 }
148 }
149 }
150
151 @Preview(showBackground = true)
152 @Composable
153 fun TipTimePreview() {
154     Modul_2Theme {
155         TipTime()
156     }
157 }
158

```

TipTimeViewModel.kt

Table 4. Source Code Jawaban Soal 1

```

1 package com.example.modul_2
2
3 import androidx.lifecycle.ViewModel
4 import kotlinx.coroutines.flow.MutableStateFlow
5 import kotlinx.coroutines.flow.StateFlow
6 import kotlin.math.ceil
7
8 class TipTimeViewModel : ViewModel() {
9
10     private val _costInput = MutableStateFlow("")
11     val costInput: StateFlow<String> = _costInput
12

```

```

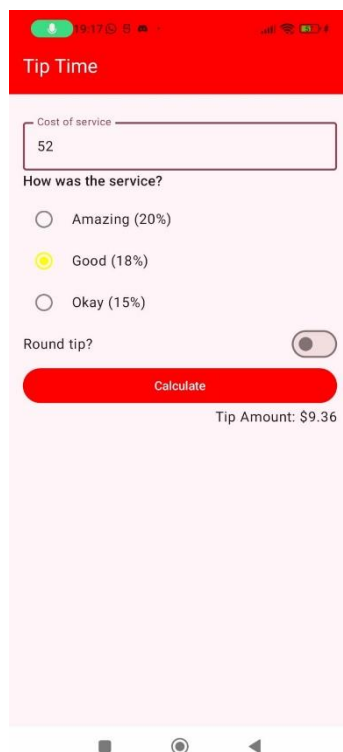
13     private val _selectedPercentage = MutableStateFlow(0.20)
14     val      selectedPercentage:      StateFlow<Double>      =
15     _selectedPercentage
16
17     private val _roundTip = MutableStateFlow(true)
18     val roundTip: StateFlow<Boolean> = _roundTip
19
20     private val _tipAmount = MutableStateFlow("Tip Amount")
21     val tipAmount: StateFlow<String> = _tipAmount
22
23     fun onCostChanged(newCost: String) {
24         _costInput.value = newCost
25     }
26
27     fun onPercentageChanged(newPercentage: Double) {
28         _selectedPercentage.value = newPercentage
29     }
30
31     fun onRoundTipChanged(newRound: Boolean) {
32         _roundTip.value = newRound
33     }
34
35     fun calculateTip() {
36         val cost = costInput.value.toDoubleOrNull()
37         if (cost != null) {
38             val tip = cost * selectedPercentage.value
39             val finalTip = if (roundTip.value) ceil(tip) else
40 tip
41             _tipAmount.value = "Tip Amount:
42 $%.2f".format(finalTip)
43         } else {
44             _tipAmount.value = "Tip Amount"
45         }
46     }
47 }

```

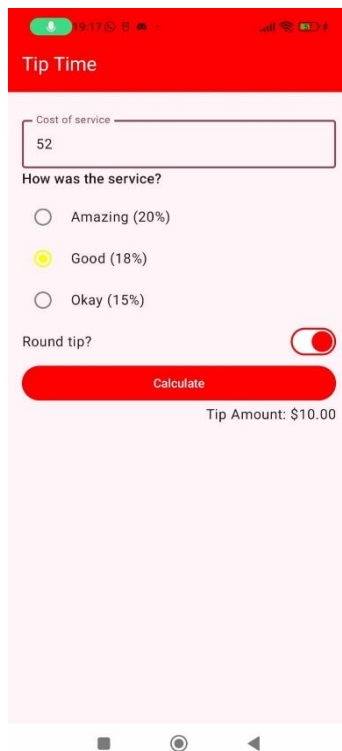
B. Output Program



Gambar 4. Screenshot Splashscreen Aplikasi



Gambar 5. Screenshot Tampilan Aplikasi



Gambar 6. Screenshot Tampilan Aplikasi



Gambar 7. Screenshot Tampilan Aplikasi

C. Pembahasan

MainActivity.kt:

Line 1, Pada line ini, dideklarasikan nama package file Kotlin com.example.modul_2

Line 3, Diimport Bundle, yang digunakan untuk menyimpan data dan mengirimkan data antar aktivitas, sering digunakan dalam metode onCreate() untuk menerima data yang dikirimkan ke aktivitas baru

Line 4, Diimport ComponentActivity dari AndroidX yang digunakan sebagai superclass untuk aktivitas yang menggunakan Jetpack Compose.

Line 5, Diimport setContent yang digunakan untuk mengonfigurasi konten UI di dalam activity

Line 6, Diimport enableEdgeToEdge, yang memungkinkan konten UI aplikasi agar dapat dilihat hingga ke tepi layar perangkat

Line 7-22, Diimport beberapa komponen layout seperti Column, Row, Spacer, fillMaxSize, padding, yang digunakan untuk pengaturan tata letak antarmuka pengguna dalam Compose

Line 23, Diimport Button dari Material3, digunakan untuk membuat tombol interaktif di UI

Line 24, Diimport ButtonDefaults, yang memungkinkan kita mengatur warna default atau gaya tombol, seperti mengubah warna latar belakang atau teks

Line 25, Diimport ExperimentalMaterial3Api sebagai anotasi yang menandakan bahwa kita menggunakan API Material3 yang masih dalam status eksperimental

Line 26, Diimport MaterialTheme, yang memungkinkan kita untuk menggunakan tema Material Design pada UI, seperti pengaturan warna dan tipografi

Line 27, Diimport OutlinedTextField, yang digunakan untuk membuat field input dengan garis tepi di sekitarnya

Line 28, Diimport RadioButtons, yang digunakan untuk menampilkan pilihan tunggal dalam bentuk tombol radio, yang hanya memungkinkan pengguna memilih satu dari beberapa opsi yang tersedia

Line 29, Diimport RadioButtonsDefaults, yang digunakan untuk mengatur warna dan warna radio button, seperti warna ketika dipilih atau tidak dipilih

Line 30, Diimport Scaffold, komponen tata letak utama di Jetpack Compose yang mendukung struktur UI seperti top app bar, floating action button (FAB), dan body konten

Line 31, Diimport Switch, yang digunakan untuk membuat komponen toggle, memungkinkan pengguna untuk memilih antara dua status (on/off)

Line 32, Diimport SwitchDefaults, untuk mengatur warna dan tampilan dari komponen switch, termasuk warna latar belakang dan thumb ketika berada dalam kondisi on/of

Line 33, Diimport Text, digunakan untuk menampilkan teks di UI Ini adalah komponen dasar untuk menampilkan string di layar

Line 34, Diimport TopAppBar, digunakan untuk menampilkan bar bagian atas aplikasi yang biasanya berisi judul aplikasi atau tindakan penting lainnya

Line 35, Diimport TopAppBarDefaults, untuk mengatur warna default dan gaya dari top app bar, seperti latar belakang dan warna teks

Line 36, Diimport Composable, adalah anotasi yang digunakan untuk menandai fungsi sebagai composable, yaitu fungsi yang mengembalikan UI dalam Jetpack Compose

Line 37, Diimport collectAsState, digunakan untuk mengubah aliran data (Flow) dari ViewModel menjadi state yang dapat dipantau di Compose

Line 38, Diimport getValue, digunakan untuk mendeklarasikan delegasi properti dengan menggunakan by, memungkinkan kita untuk mengakses nilai property secara otomatis

Line 39, Diimport Alignment, digunakan untuk mengatur posisi elemen dalam layout, baik secara vertikal maupun horizontal

Line 40, Diimport Modifier, digunakan untuk menambahkan atau memodifikasi properti dari elemen UI seperti ukuran, padding, margin, dan lain-lain

Line 41, Diimport Color, digunakan untuk mengatur warna elemen UI seperti latar belakang, teks, tombol, dan lain-lain

Line 42, Diimport TextAlign, digunakan untuk mengatur perataan teks secara horizontal di dalam komponen Text

Line 43, Diimport tooling.preview.Preview, digunakan untuk menampilkan preview dari UI Compose dalam Android Studio sebelum dijalankan pada perangkat

Line 44, Diimport `unit.dp`, digunakan untuk menentukan ukuran dalam satuan density-independent pixels (dp), yang berguna untuk membuat UI responsif di berbagai ukuran layar

Line 45, Diimport `installSplashScreen`, digunakan untuk mengonfigurasi splash screen pada aplikasi Android yang muncul saat aplikasi dijalankan

Line 46, Diimport `viewModel`, digunakan untuk mengakses instance dari `ViewModel` untuk memisahkan logika bisnis dan UI dalam aplikasi

Line 47, Diimport `rememberScrollState`, digunakan untuk membuat state yang mengingat status scroll saat pengguna menggulir layout

Line 48, Diimport `verticalScroll`, digunakan untuk memberikan kemampuan scroll vertikal pada layout, memungkinkan pengguna untuk menggulir konten

Line 49, Diimport `Modul_2Theme`, adalah tema kustom aplikasi yang diterapkan ke seluruh tampilan antarmuka

Line 51, Di dalam `MainActivity`, kita mendefinisikan fungsi `onCreate()` Fungsi ini dijalankan saat aktivitas pertama kali dibuat dan digunakan untuk mengonfigurasi konten UI dan pengaturan lainnya

Line 52, Dalam fungsi `onCreate()`, kita memanggil `installSplashScreen()` untuk menampilkan layar pembuka saat aplikasi mulai Fungsi ini menyediakan efek loading sementara aplikasi berjalan

Line 53, Selanjutnya, `enableEdgeToEdge()` digunakan untuk membuat layout aplikasi bisa merentang hingga ke tepi layar perangkat, memberikan tampilan modern dengan edge-to-edge design

Line 54, `setContent {}` adalah fungsi `Compose` yang menggantikan `setContentView()` Di sini, kita menggunakan tema kustom `Modul_2Theme` dan menyetel UI yang akan ditampilkan di aktivitas, yaitu fungsi `TipTime()`

Line 56, Fungsi `TipTime()` adalah composable yang digunakan untuk membuat UI dari aplikasi Fungsi ini mengelola semua elemen interaktif, seperti inputan pengguna dan tampilan hasil kalkulasi

Line 57, `viewModel()` digunakan untuk mendapatkan instance dari `TipTimeViewModel`, yang menyimpan data dan logika bisnis dari aplikasi

Line 58-60, Mendeklarasikan variabel dengan `collectAsState()`, yang digunakan untuk mengambil nilai terbaru dari aliran data yang dikendalikan oleh `ViewModel`. Variabel-variabel ini berisi nilai yang berkaitan dengan input pengguna seperti biaya layanan, pilihan persentase tip, apakah tip dibulatkan, dan jumlah tip yang dihitung. Line 62, `radioOptions` adalah daftar pasangan label teks dan nilai persentase yang digunakan dalam pilihan radio button untuk memilih tingkat pelayanan (misalnya, Amazing 20%)

Line 64-72, Scaffold digunakan untuk membuat layout dasar dengan top app bar dan konten utama aplikasi. Di dalam Scaffold, kita mendeklarasikan semua elemen UI yang ditampilkan di layar, seperti field input, pilihan radio button, switch, tombol, dan teks hasil perhitungan.

Line 74-82, Bagian ini berisi elemen UI seperti `OutlinedTextField` untuk input biaya layanan, teks penjelasan, `RadioButton` untuk memilih tingkat pelayanan, dan `Switch` untuk memilih apakah tip dibulatkan.

Line 84-86, Button digunakan untuk menghitung tip saat pengguna menekan tombol, di mana `viewModel::calculateTip` dipanggil untuk menghitung nilai tip berdasarkan input pengguna.

Line 88-90, Text digunakan untuk menampilkan jumlah tip yang dihitung, dengan perataan teks ke kanan (`end`) menggunakan `TextAlign.End`.

Line 92-94, `TipTimePreview` adalah fungsi anotasi `@Preview` yang memungkinkan kita melihat tampilan `TipTime()` secara langsung di Android Studio tanpa menjalankan aplikasi pada perangkat. Ini memberikan gambaran awal dari UI tanpa perlu memuat aplikasi sepenuhnya.

TipTimeViewModel.kt

Line 1, Mendeklarasikan nama package `com.example.modul_2` sebagai lokasi file dalam struktur proyek aplikasi.

Line 3, Mengimpor `ViewModel` dari `AndroidX` untuk mengelola data UI.

Line 4, Mengimpor `MutableStateFlow` dan `StateFlow` dari `Kotlin Coroutines` untuk pengelolaan dan pemantauan data secara reaktif.

Line 5, Mengimpor fungsi `ceil` dari `kotlin.math` untuk membulatkan nilai ke atas.

Line 7, Mendeklarasikan kelas TipTimeViewModel yang merupakan turunan dari ViewModel, berfungsi untuk mengelola data kalkulasi tip

Line 9, Mendeklarasikan variabel `_costInput` bertipe `MutableStateFlow<String>` untuk menyimpan input biaya layanan dari pengguna

Line 10, Mendeklarasikan properti `costInput` bertipe `StateFlow<String>` untuk memberikan akses hanya-baca terhadap nilai `_costInput`

Line 12, Mendeklarasikan variabel `_selectedPercentage` bertipe `MutableStateFlow<Double>` untuk menyimpan persentase tip yang dipilih

Line 13, Mendeklarasikan properti `selectedPercentage` bertipe `StateFlow<Double>` untuk memberikan akses hanya-baca terhadap nilai persentase tip

Line 15, Mendeklarasikan variabel `_roundTip` bertipe `MutableStateFlow<Boolean>` untuk menyimpan status pembulatan tip

Line 16, Mendeklarasikan properti `roundTip` bertipe `StateFlow<Boolean>` untuk memberikan akses hanya-baca terhadap status pembulatan

Line 18, Mendeklarasikan variabel `_tipAmount` bertipe `MutableStateFlow<String>` untuk menyimpan hasil kalkulasi jumlah tip

Line 19, Mendeklarasikan properti `tipAmount` bertipe `StateFlow<String>` untuk memberikan akses hanya-baca terhadap nilai hasil kalkulasi tip

Line 21, Mendeklarasikan fungsi `onCostChanged(newCost: String)` untuk memperbarui nilai `_costInput` dengan input baru dari pengguna

Line 23, Mendeklarasikan fungsi `onPercentageChanged(newPercentage: Double)` untuk memperbarui nilai `_selectedPercentage`

Line 25, Mendeklarasikan fungsi `onRoundTipChanged(newRound: Boolean)` untuk memperbarui nilai `_roundTip`

Line 27, Mendeklarasikan fungsi `calculateTip()` untuk menghitung jumlah tip berdasarkan input biaya dan persentase tip

Line 28, Mengambil nilai `cost` dari `costInput` dan mengonversinya menjadi `Double`, jika tidak valid maka hasil tip tidak diperbarui

Line 30, Mengalikan biaya layanan dengan persentase tip untuk mendapatkan nilai tip awal

Line 32, Membulatkan nilai tip menggunakan `ceil` jika status pembulatan diaktifkan

Line 33, Memformat hasil kalkulasi tip dan menyimpannya dalam `_tipAmount` dalam format mata uang.

MODUL 3 : Build a Scrollable List

SOAL 1

Soal Praktikum:

Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut: a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

A. Source Code

MainActivity.kt

Table 5. Source Code MainActivity.kt

1	package com.example.modul_3
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.runtime.Composable
7	import androidx.compose.ui.tooling.preview.Preview

8	import com.example.modul_3.navigation.NavGraph
9	
10	class MainActivity : ComponentActivity() {
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	setContent {
14	NavGraph()
15	}
16	}
17	}
18	
19	@Preview(showBackground = true)
20	@Composable
21	fun Preview() {
22	NavGraph()
23	}

DataTournament.kt

Table 6. Source Code DataTournament.kt

1	package com.example.modul_3.data
2	
3	data class ListItem(
4	val title: String, val desc: String, val imageUrl: String,
5	val linkUrl: String)
6	
7	val sampleItems = listOf(
8	ListItem("Indonesia Open", "Indonesia Open adalah
9	sebuah kejuaraan bulu tangkis internasional di Indonesia.
10	Kejuaraan ini diselenggarakan oleh Persatuan Bulu Tangkis
11	Seluruh Indonesia dengan edisi terkini berlangsung di
12	Istora Gelora
13	Bung Karno, Jakarta.", "https://img.bwfbadminton.com/image/u
14	pload/v1746_607228/assets/tournaments/logo/8E66BAA2-CF0F
15	-4FFA-8798-FF700D1CDBEC.png", "https://bwfworldtour.bwfb
16	adminton.com/tournament/5236/indonesia-open-2025/results/2
17	025-06-03"),
18	ListItem("Yonex All England Open", "Turnamen bulutangkis
19	tertua di dunia yang diadakan di Inggris sejak 1899.
20	Saat ini menjadi bagian dari BWF World Tour Super 1000
21	dan sangat bergengsi karena Sejarah
22	dan prestisenya.", "https://img.bwfbadminton.com/image
23	/upload
24	/v1734076435/assets/tournaments/logo/F6304F7C-D298-4E3D
25	AA62A6B933CBB8C7.png", "https://bwfworldtour.bwfbadminton.com/
26	tournament/5238/yonex-all-england-open-badminton-champions
27	hips-2025/results/podium/"),

28	ListItem("Victor China Open", "Turnamen bulutangkisintern
29	asional bergengsi yang diadakan setiap tahun di Tiongkok.
30	Merupakan bagian dari BWF World Tour Super 1000 (bersama
31	All England dan Indonesia Open).
32	Menarik banyak pemain top dunia karena
33	hadiah besar dan poin peringkat tinggi.", "https://img.bw
34	fbadminton.com/image/upload/v17
35	46149516/assets/tournaments/logo/5A59A1E7-1366-49BA-863D
36	-33FB844D51C0.png", "https://bwfworldtour.bwfbadm
37	ton.com/tournament/5281/victor-china-open-2025/results
38	/2025-07-22"),
39	ListItem("Daihatsu Indonesia Master", "Indonesia Mast
40	er adalah sebuah kejuaraan bulu tangkis internasional
41	di Indonesia. Kejuaraan ini diselenggarakan oleh
42	Persatuan Bulu Tangkis Seluruh Indonesia dengan edisi
43	terkini berlangsung di
44	Istora Gelora
45	Bung
46	Karno, Jakarta.", "https://img.bwfbadminton.com/image/
47	upload/
48	v1731292348/assets/tournaments/logo/C166BF11-2128-4F8C-B1ED-
49	F30565DE9089.png",
50	"https://bwfworldtour.bwfbadminton.com/tournament/5234/
51	daihatsu-indonesia-masters-2025/results/podium/"),
52	ListItem("HSBC BWF World Tour Final", "Turnamen penutup
53	musim bulutangkis yang hanya mempertemukan 8 pemain atau
54	pasangan terbaik dunia dari setiap sektor berdasarkan
55	perolehan
56	poin sepanjang musim BWF World Tour. Turnamen ini sangat
57	bergengsi dan berhadiah
58	besar, setara dengan kejuaraan elit lainnya.", "
59	https://img.bwfbadminton.com/image/
60	upload/v1734514049
61	/assets/tournaments/logo/F34EBDDDB-AA5D-4F1E-A14F
62	-A94473DD1
63	295.png", "https://bwfworldtourfinals.bwfbadm
64	inton.com/results/5259/hsbc-bwf-world-tour-finals-
65	2025/draw/"),
66	ListItem("TotalEnergies BWF Sudirman Cup Finals",
67	"Sudirman Cup adalah kejuaraan bulu tangkis
68	
69	internasional untuk nomor beregu campuran,
70	mempertandingkan nomor tunggal putra, tunggal putri, ganda
71	putra, ganda putri, dan ganda campuran.
72	Kejuaraan ini digelar setiap dua tahun
73	sekali.", "https://img.bwfbadminton.com/image/upload/v
74	1734511970/assets/tournaments/logo/C9F72B3B-2C6A-4D63-

75	A782-78968933BDE9.png", "https://bwfsudirmancup.bwfb
76	adminton.com/results/5260/totalenergies-bwf-sudirman-cu
77	p-finals-2025/podium"),
78	ListItem("TotalEnergies BWF Sudirman Cup Finals",
79	"Sudirman Cup adalah kejuaraan bulu tangkis inte
80	rnasional untuk nomor beregu campuran, mempertandingkan
81	nomor tunggal putra, tunggal putri, ganda putra, ganda
82	putri, dan ganda campuran. Kej
83	uaaraan ini digelar setiap dua tahun sekali.", "https://
84	img.bwfbadminton.com/image/upload/v1
85	734511970/assets/tournaments/logo/C9F72B3B-2C6A-4D63-A782-
86	78968933BDE9.png", "https://bwfsudirmancup.bwfbadm
87	inton.com/results/5260/totalenergies-bwf-sudirman-cup-finals
88	-2025/podium"),
89	ListItem("TotalEnergies BWF Thomas & Uber Cup Finals",
90	"Thomas Cup dan Uber Cup adalah kejuaraan bulutangkis
91	beregu putra dan putri antarnegara yang diselenggarakan
92	dua tahun sekali oleh BWF. Setiap tim terdiri dari pemain
93	tunggal dan ganda, dan bertanding untuk menjadi tim
94	nasional putra dan putri terbaik di dunia",
95	"https://bwfthomasubercups.bwfbadminton.com/wp-conten
96	t/plugins/bwf-menu-
97	system/images/TUCTournamentLogoLandscape.png",
98	"https://bwfthomasubercups.bwfbadminton.com/results/5
99	147/totalenergies-bwf-thomas-uber-cup-finals-2024/podium"),
100	)

NavGraph.kt

Table 7. Source Code NaavGraph.kt

1	package com.example.modul_3.navigation
2	
3	import androidx.compose.runtime.Composable
4	import androidx.navigation.NavType
5	import androidx.navigation.compose.NavHost
6	import androidx.navigation.compose.composable
7	import androidx.navigation.compose.rememberNavController
8	import androidx.navigation.navArgument
9	import com.example.modul_3.screen.DetailScreen
10	import com.example.modul_3.screen.ListScreen
11	
12	@Composable
13	fun NavGraph() {
14	val navController = rememberNavController()
15	
16	

17	NavHost(navController	=	navController,
18	startDestination = "list") {		
19	composable("list") {		
20	ListScreen(navController)		
21	}		
22	composable(		
23	"detail/{itemTitle}",		
24	arguments = listOf(navArgument("itemTitle") {		
25	type = NavType.StringType })		
26	) { backStackEntry ->		
27	val	itemTitle	=
28	backStackEntry.arguments?.getString("itemTitle")	?:	"No
29	Title"		
30	DetailScreen(itemTitle)		
31	}		
32	}		
33	}		

ListScreen.kt

Table 8. Souce Code ListScreen.kt

1	package com.example.modul_3.screen
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.layout.Arrangement
6	import androidx.compose.foundation.layout.Column
7	import androidx.compose.foundation.layout.Row
8	import androidx.compose.foundation.layout.Spacer
9	import androidx.compose.foundation.layout.fillMaxSize
10	import androidx.compose.foundation.layout.fillMaxWidth
11	import androidx.compose.foundation.layout.padding
12	import androidx.compose.foundation.layout.size
13	import androidx.compose.foundation.layout.width
14	import androidx.compose.foundation.lazy.LazyColumn
15	import androidx.compose.foundation.lazy.items
16	import
17	androidx.compose.foundation.shape.RoundedCornerShape
18	import androidx.compose.material3.Button
19	import androidx.compose.material3.Card
20	import androidx.compose.material3.MaterialTheme
21	import androidx.compose.material3.Text
22	import androidx.compose.runtime.Composable
23	import androidx.compose.ui.Alignment
24	import androidx.compose.ui.Modifier
25	import androidx.compose.ui.draw.clip
26	import androidx.compose.ui.layout.ContentScale

```

27 import androidx.compose.ui.platform.LocalContext
28 import androidx.compose.ui.unit.dp
29 import androidx.navigation.NavController
30 import coil.compose.AsyncImage
31 import com.example.modul_3.data.sampleItems
32
33 @Composable
34 fun ListScreen(navController: NavController) {
35     val context = LocalContext.current
36
37     LazyColumn(
38         modifier = Modifier
39             .fillMaxSize()
40             .padding(8.dp)
41     ) {
42         items(sampleItems) { item ->
43             Card(
44                 shape = RoundedCornerShape(16.dp),
45                 modifier = Modifier
46                     .fillMaxWidth()
47                     .padding(8.dp)
48             ) {
49                 Row(
50                     modifier = Modifier
51                         .padding(16.dp)
52                         .fillMaxWidth(),
53                     verticalAlignment =
54 Alignment.CenterVertically
55                 ) {
56                     AsyncImage(
57                         model = item.imageUrl,
58                         contentDescription = item.title,
59                         modifier = Modifier
60                             .size(100.dp)
61
62                     .clip(RoundedCornerShape(12.dp)),
63                     contentScale = ContentScale.Crop
64                 )
65
66                     Spacer(modifier =
67 Modifier.width(16.dp))
68
69                     Column(
70                         modifier = Modifier.weight(1f),
71                         verticalArrangement =
72 Arrangement.spacedBy(8.dp)
73                     ) {

```

74	Text(text = item.title, style =
75	MaterialTheme.typography.titleMedium)
76	Text(text = item.desc, style =
77	MaterialTheme.typography.bodySmall)
78	
79	Row(
80	horizontalArrangement =
81	Arrangement.spacedBy(8.dp)
82	) {
83	Button(onClick = {
84	val intent =
85	Intent(Intent.ACTION_VIEW, Uri.parse(item.linkUrl)).apply
86	{
87	
88	addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
89	}
90	
91	context.startActivity(intent)
92	)) {
93	Text("BWF")
94	}
95	
96	Button(onClick = {
97	
98	navController.navigate("detail/\${item.title}")
99	)) {
100	Text("Detail")
101	}
102	}
103	}
104	}
105	}
106	}
107	}
108	}

DetailScreen.kt

Table 9. Source Code DetailScreen.kt

1	package com.example.modul_3.screen
2	
3	import androidx.compose.foundation.layout.Box
4	import androidx.compose.foundation.layout.Column
5	import androidx.compose.foundation.layout.Spacer
6	import androidx.compose.foundation.layout.fillMaxSize
7	import androidx.compose.foundation.layout.fillMaxWidth

```

8 import androidx.compose.foundation.layout.height
9 import androidx.compose.foundation.layout.padding
10 import androidx.compose.foundation.layout.wrapContentSize
11 import
12 androidx.compose.foundation.shape.RoundedCornerShape
13 import androidx.compose.material3.Card
14 import androidx.compose.material3.MaterialTheme
15 import androidx.compose.material3.Text
16 import androidx.compose.runtime.Composable
17 import androidx.compose.ui.Alignment
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.draw.clip
20 import androidx.compose.ui.graphics.Color
21 import androidx.compose.ui.layout.ContentScale
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.unit.dp
24 import coil.compose.AsyncImage
25 import com.example.modul_3.data.sampleItems
26
27 @Composable
28 fun DetailScreen(itemTitle: String) {
29     val item = sampleItems.firstOrNull { it.title ==
30 itemTitle }
31
32     item?.let {
33         Box(
34             contentAlignment = Alignment.Center,
35             modifier
36 Modifier.fillMaxSize().padding(16.dp)
37         ) {
38             Column(horizontalAlignment
39 Alignment.CenterHorizontally) {
40                 Card(
41                     shape = RoundedCornerShape(16.dp),
42                     modifier = Modifier
43                         .fillMaxWidth()
44                         .height(250.dp)
45                         .clip(RoundedCornerShape(16.dp))
46                 ) {
47                     AsyncImage(
48                         model = it.imageUrl,
49                         contentDescription = itemTitle,
50                         modifier = Modifier
51                             .fillMaxSize()
52
53                     .clip(RoundedCornerShape(16.dp)),
54                     contentScale = ContentScale.Crop

```

```

55         )
56     }
57
58     Spacer(modifier = Modifier.height(24.dp))
59
60     Text(
61         text = it.title,
62         style =
63     MaterialTheme.typography.titleMedium.copy(
64         fontWeight = FontWeight.Bold,
65         color =
66     MaterialTheme.colorScheme.primary
67     ),
68     modifier = Modifier
69         .fillMaxWidth()
70         .padding(horizontal = 16.dp)
71
72     .align(Alignment.CenterHorizontally)
73     )
74
75     Spacer(modifier = Modifier.height(8.dp))
76
77     Text(
78         text = "Description:",
79         style =
80     MaterialTheme.typography.bodyMedium.copy(
81         fontWeight = FontWeight.SemiBold,
82         color =
83     MaterialTheme.colorScheme.secondary
84     ),
85     modifier =
86     Modifier.padding(horizontal = 16.dp)
87     )
88
89     Text(
90         text = it.desc,
91         style =
92     MaterialTheme.typography.bodyLarge,
93     modifier =
94     Modifier.padding(horizontal = 16.dp)
95     )
96     }
97 }
98 } ?: run {
99     Text(
100         text = "Item tidak ditemukan",
101

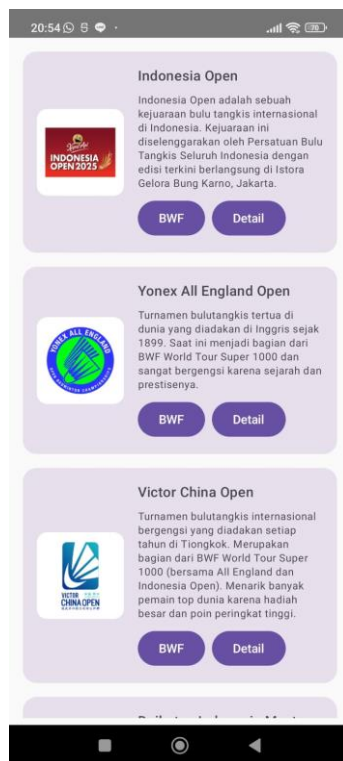
```

```

102         style
103     MaterialTheme.typography.bodyLarge.copy(
104         fontWeight = FontWeight.Bold,
105         color = Color.Red
106     ),
107     modifier = Modifier
108         .fillMaxSize()
109         .wrapContentSize(Alignment.Center)
110     )
111 }

```

B. Output Program



Gambar 8. Screenshot Halaman Awal



Gambar 9. Screenshot Ketika Tombol BWF Ditekan Mengarah Ke Website



Gambar 10. Screenshot Ketika Tombol Detail Ditekan

C. Pembahasan

MainActivity.kt:

Line 1, Mendeklarasikan nama package `com.example.modul_3` sebagai struktur direktori tempat file berada dalam proyek

Line 3, Mengimpor `Bundle` dari `android.os` untuk menyimpan dan mengirim data antar komponen saat aktivitas dibuat atau dijalankan kembali

Line 4, Mengimpor `ComponentActivity` dari `androidx.activity` sebagai kelas dasar untuk aktivitas yang menggunakan Jetpack Compose

Line 5, Mengimpor fungsi `setContent` dari `androidx.activity.compose` untuk menetapkan isi UI aktivitas menggunakan deklarasi Compose

Line 6, Mengimpor anotasi `@Composable` dari `androidx.compose.runtime` untuk menandai fungsi sebagai fungsi komposisi dalam Jetpack Compose

Line 7, Mengimpor anotasi `@Preview` dari `androidx.compose.ui.tooling.preview` untuk menampilkan pratinjau komponen Compose di Android Studio

Line 8, Mengimpor `NavGraph` dari `com.example.modul_3.navigation` yang berisi pengaturan navigasi antar layar dalam aplikasi

Line 10, Mendeklarasikan kelas `MainActivity` sebagai turunan dari `ComponentActivity`, menjadi entry point utama aplikasi

Line 11, Fungsi `onCreate` dioverride dari `ComponentActivity` dan dipanggil saat aktivitas pertama kali dibuat

Line 12, Memanggil `super.onCreate(savedInstanceState)` untuk memastikan inisialisasi komponen aktivitas dari superclass

Line 13, Menggunakan fungsi `setContent` untuk menetapkan isi UI menggunakan Jetpack Compose

Line 14, Memanggil `MainActivity`

Line 19, Anotasi `@Preview(showBackground = true)` digunakan untuk menampilkan pratinjau UI dengan latar belakang putih

Line 20, Mendeklarasikan fungsi `Preview()` sebagai fungsi Compose untuk menampilkan pratinjau dari `NavGraph` di Android Studio

Line 21, Fungsi `NavGraph()` dipanggil di dalam fungsi `Preview` agar struktur navigasi bisa dilihat dalam mode pratinjau

Line 22, Kurung kurawal penutup untuk menutup fungsi Preview

DataTournament.kt

Line 1, Mendeklarasikan nama package `com.example.modul_3.data` sebagai lokasi file dalam struktur proyek

Line 3, Mendefinisikan data class `ListItem` dengan empat properti: `title`, `desc`, `imageUrl`, dan `linkUrl` yang bertipe `String` untuk merepresentasikan data turnamen

Line 5, Mendeklarasikan variabel `sampleItems` bertipe `List` yang berisi kumpulan objek `ListItem`

Line 6-14, Menambahkan objek `ListItem` pertama dengan informasi tentang turnamen Indonesia Open meliputi judul, deskripsi, URL gambar, dan link situs

Line 15-21, Menambahkan objek `ListItem` kedua tentang Yonex All England Open dengan data lengkap seperti objek sebelumnya

Line 22-28, Menambahkan objek `ListItem` ketiga tentang Victor China Open, turnamen bergengsi di Tiongkok

Line 29-35, Menambahkan objek `ListItem` keempat mengenai Daihatsu Indonesia Master yang diselenggarakan di Jakarta

Line 36-42, Menambahkan objek `ListItem` kelima tentang HSBC BWF World Tour Final, turnamen penutup musim untuk delapan pemain atau pasangan terbaik

Line 43-49, Menambahkan objek `ListItem` keenam mengenai TotalEnergies BWF Sudirman Cup Finals, turnamen beregu campuran

Line 50-56, Menambahkan kembali objek `ListItem` yang identik dengan sebelumnya yaitu tentang TotalEnergies BWF Sudirman Cup Finals

Line 57-63, Menambahkan objek `ListItem` kedelapan tentang TotalEnergies BWF Thomas & Uber Cup Finals, kejuaraan beregu putra dan putri antarnegara

NavGraph.kt

Line 1, Mendeklarasikan nama package `com.example.modul_3.navigation` sebagai lokasi file dalam struktur proyek

Line 3, Mengimpor `Composable` dari `androidx.compose.runtime` untuk menandai fungsi sebagai bagian dari UI deklaratif Jetpack Compose

Line 4, Mengimpor NavType dari androidx.navigation untuk menentukan tipe data dari argumen navigasi

Line 5, Mengimpor NavHost dari androidx.navigation.compose sebagai container utama untuk mendefinisikan rute navigasi

Line 6, Mengimpor composable dari androidx.navigation.compose untuk mendefinisikan layar yang dapat dinavigasikan dalam NavHost

Line 7, Mengimpor rememberNavController untuk membuat dan mengingat instance NavController sebagai pengatur navigasi

Line 8, Mengimpor navArgument dari androidx.navigation untuk mendefinisikan argumen yang digunakan pada rute tertentu

Line 9, Mengimpor DetailScreen dari package com.example.modul_3.screen sebagai layar detail suatu item

Line 10, Mengimpor ListScreen dari package com.example.modul_3.screen sebagai layar daftar awal item

Line 12, Mendeklarasikan fungsi NavGraph() dengan anotasi @Composable untuk membangun UI menggunakan Jetpack Compose

Line 13, Membuat variabel navController menggunakan rememberNavController() untuk mengatur navigasi

Line 15, Menggunakan NavHost untuk menentukan rute navigasi dengan navController dan startDestination yang mengarah ke "list"

Line 16, Mendefinisikan rute composable("list") untuk memanggil fungsi ListScreen saat rute ini aktif

Line 17, Memanggil fungsi ListScreen(navController) untuk menampilkan daftar item dengan dukungan navigasi

Line 18, Mendefinisikan rute composable("detail/{itemTitle}") dengan parameter itemTitle yang diterima dari layar sebelumnya

Line 19, Menggunakan navArgument("itemTitle") untuk mendeklarasikan argumen bertipe String yang diperlukan oleh rute detail

Line 20, Menandai blok lambda dari rute detail untuk menerima backStackEntry sebagai data dari rute sebelumnya

Line 21, Mengambil nilai dari argumen `itemTitle` melalui `backStackEntry` atau menggunakan nilai default "No Title" jika tidak ditemukan

Line 22, Memanggil fungsi `DetailScreen(itemTitle)` untuk menampilkan layar detail berdasarkan argumen yang diterima

ListScreen.kt

Pada line 1, dideklarasikan nama package `com.example.modul_3.screen` sebagai lokasi file ini berada dalam struktur proyek.

Pada line 3, diimpor `Intent` dari `Android` untuk membuat aksi berpindah ke aplikasi lain (misal membuka URL di browser).

Pada line 4, diimpor `Uri` dari `Android` untuk menangani data URI pada `Intent`.

Pada line 5–14, diimpor berbagai komponen layout dari `Jetpack Compose` seperti `Column`, `Row`, `Spacer`, `LazyColumn`, dll, yang digunakan untuk menyusun tampilan UI secara deklaratif.

Pada line 15, diimpor `RoundedCornerShape` untuk membuat sudut komponen berbentuk membulat.

Pada line 16–18, diimpor komponen `Material3` seperti `Button`, `Card`, dan `Text` untuk membangun antarmuka material design.

Pada line 19, diimpor anotasi `@Composable` agar fungsi dapat digunakan sebagai komponen UI dalam `Compose`.

Pada line 20–24, diimpor utilitas `Compose` tambahan, seperti `Modifier`, `Alignment`, `ContentSize`, dan unit pengukuran `dp`.

Pada line 25, diimpor `LocalContext` untuk mengakses `Context` dari `Android` dalam komposisi.

Pada line 26, diimpor `NavController` dari `androidx.navigation` untuk menangani navigasi antar layar.

Pada line 27, diimpor `AsyncImage` dari library `Coil` untuk memuat gambar dari URL secara asynchronous.

Pada line 28, diimpor `sampleItems` dari package `com.example.modul_3.data` sebagai data yang akan ditampilkan dalam daftar.

Pada line 30, dideklarasikan fungsi `ListScreen(navController: NavController)` sebagai layar daftar yang bisa dinavigasikan dan menerima parameter `NavController`. Pada line 31, dibuat variabel `context` dari `LocalContext.current` untuk mendapatkan `context` saat ini dari komposisi.

Pada line 33, digunakan `LazyColumn` sebagai komponen `scrollable` untuk menampilkan daftar secara efisien.

Pada line 34–36, diberikan `Modifier` untuk mengisi seluruh ukuran dan memberikan `padding 8dp` ke `LazyColumn`.

Pada line 37, digunakan fungsi `items(sampleItems)` untuk mengiterasi semua data dari `sampleItems`.

Pada line 38, dideklarasikan `Card` untuk setiap item yang dibungkus dalam desain kartu.

Pada line 39–41, diberikan bentuk sudut membulat dan `padding` pada `Card` agar tampilannya lebih rapi dan modern.

Pada line 42, dideklarasikan `Row` untuk menampilkan gambar dan teks secara horizontal sejajar.

Pada line 43–45, digunakan `Modifier` untuk `padding` dan memenuhi lebar maksimum, serta menyetel perataan vertikal ke tengah (`Alignment.CenterVertically`).

Pada line 46, digunakan `AsyncImage` untuk memuat gambar dari URL item.

Pada line 47–51, diberikan atribut model gambar, deskripsi konten, ukuran, `clipping` bundar, dan skala `crop` agar gambar tampil dengan proporsi yang bagus.

Pada line 53, digunakan `Spacer` untuk memberi jarak horizontal antara gambar dan teks.

Pada line 55, dibuat `Column` untuk menampilkan teks dan tombol secara vertikal.

Pada line 56, digunakan `Modifier.weight(1f)` agar kolom mengambil sisa ruang, dan `Arrangement.spacedBy(8.dp)` untuk memberi jarak antar elemen di dalam kolom.

Pada line 57, ditampilkan judul item dengan style `titleMedium` dari `MaterialTheme`.

Pada line 58, ditampilkan deskripsi item dengan style `bodySmall`.

Pada line 60, dibuat `Row` lagi untuk menampung dua tombol dalam satu baris horizontal.

Pada line 61, digunakan `Arrangement.spacedBy(8.dp)` agar tombol memiliki jarak antar satu sama lain.

Pada line 62–66, tombol "BWF" dibuat, yang ketika ditekan akan membuka link URL menggunakan `Intent.ACTION_VIEW`.

Pada line 67, tombol "Detail" dibuat, yang ketika ditekan akan berpindah ke layar detail berdasarkan judul item.

Pada line 68, menggunakan `navController.navigate("detail/${item.title}")` untuk melakukan navigasi dan mengirim judul item sebagai argumen.

Pada line 69, teks tombol "Detail" ditampilkan.

DetailScreen.kt

Pada line 1, dideklarasikan nama package `com.example.modul_3.screen` sebagai lokasi file ini berada dalam struktur proyek.

Pada line 3–19, diimpor berbagai komponen dari Jetpack Compose seperti `Box`, `Column`, `Spacer`, `Card`, dan lain-lain yang akan digunakan untuk layout dan desain UI.

Pada line 20, diimpor `AsyncImage` dari Coil untuk menampilkan gambar secara asynchronous.

Pada line 21, diimpor `sampleItems` dari package `com.example.modul_3.data`, yang berisi data item yang akan ditampilkan pada layar detail.

Pada line 23, dideklarasikan fungsi `DetailScreen(itemTitle: String)` yang menerima parameter `itemTitle` untuk mencari item sesuai judulnya.

Pada line 24, digunakan `sampleItems.firstOrNull { it.title == itemTitle }` untuk mencari item dengan judul yang sesuai dalam daftar `sampleItems`. Jika tidak ditemukan, item akan bernilai `null`.

Pada line 26–27, menggunakan `item?.let { ... }` untuk memeriksa apakah item ditemukan. Jika ditemukan, blok kode di dalam `let` akan dieksekusi.

Pada line 28, menggunakan `Box` untuk menyusun konten secara pusat, dengan `contentAlignment = Alignment.Center` untuk memastikan semua konten berada di tengah layar. `Modifier.fillMaxSize().padding(16.dp)` digunakan agar `Box` mengisi seluruh layar dengan padding 16dp.

Pada line 30–31, menggunakan Column untuk menampilkan elemen-elemen secara vertikal dengan perataan horizontal di tengah.

Pada line 32, mendeklarasikan Card untuk menampilkan gambar dengan bentuk sudut membulat dan ukuran tinggi 250dp.

Pada line 34, digunakan AsyncImage untuk memuat gambar dari URL yang ada pada item. ContentScale.Crop digunakan untuk memotong gambar agar mengisi ukuran yang tersedia.

Pada line 38, digunakan Spacer(modifier = Modifier.height(24.dp)) untuk memberi jarak vertikal sebesar 24dp setelah gambar.

Pada line 40–41, digunakan Text untuk menampilkan judul item dengan gaya MaterialTheme.typography.titleMedium, memberi bobot font Bold, dan warna primer dari tema material.

Pada line 43, digunakan Spacer(modifier = Modifier.height(8.dp)) untuk memberi jarak vertikal setelah judul item.

Pada line 45, digunakan Text untuk menampilkan label "Description:" dengan gaya MaterialTheme.typography.bodyMedium, bobot font SemiBold, dan warna sekunder dari tema material.

Pada line 47, digunakan Text untuk menampilkan deskripsi item, menggunakan gaya MaterialTheme.typography.bodyLarge.

Pada line 50, digunakan } untuk menutup blok item?.let { ... } yang berarti tampilan detail sudah selesai ditampilkan.

Pada line 52, digunakan blok ?: run { ... } untuk menangani kasus ketika item tidak ditemukan. Jika item tidak ditemukan, akan menampilkan teks "Item tidak ditemukan".

Pada line 54, digunakan Text untuk menampilkan pesan error dengan gaya MaterialTheme.typography.bodyLarge, bobot font Bold, dan warna merah, serta memastikan teks ditampilkan di tengah layar dengan Modifier.fillMaxSize().wrapContentSize(Alignment.Center).

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Jawaban

Pertama, banyak proyek Android masih menggunakan View System, terutama aplikasi yang sudah lama dikembangkan, sehingga migrasi ke Jetpack Compose belum menjadi prioritas. RecyclerView sudah lama menjadi komponen andalan dan memiliki ekosistem pendukung yang sangat matang, seperti integrasi dengan DiffUtil, Paging, dan berbagai LayoutManager yang memberikan fleksibilitas tinggi dalam menampilkan data dalam berbagai format. Selain itu, RecyclerView memberikan kontrol dan kustomisasi lebih banyak terhadap perilaku tampilan, animasi, dan performa, yang belum sepenuhnya bisa digantikan oleh LazyColumn dalam beberapa kasus. Jetpack Compose sendiri masih dalam tahap penyempurnaan dan meskipun produktivitas lebih tinggi dalam pengembangan UI, penggunaannya lebih cocok untuk proyek baru yang sepenuhnya mengadopsi arsitektur deklaratif.

MODUL 4 : ViewModel and Debugging

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory dalam pembuatan ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

MainActivity.kt

Table 10. Source Code MainActivity.kt

1	package com.example.modul4
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.material3.MaterialTheme
7	import androidx.compose.material3.Surface
8	import androidx.navigation.compose.rememberNavController
9	import com.example.modul4.navigation.AppNavGraph
10	
11	class MainActivity : ComponentActivity() {

```

12         override fun onCreate(savedInstanceState: Bundle?) {
13             super.onCreate(savedInstanceState)
14             setContent {
15                 Surface(color                                     =
16                 MaterialTheme.colorScheme.background) {
17                     val                                     navController =
18                     rememberNavController()
19                     AppNavGraph(navController                 =
20                     navController)
21                 }
22             }
23         }
24     }
25 }

```

DataTournament.kt

Table 11. Source Code DataTournament.kt

```

1 package com.example.modul4.data
2 import com.example.modul4.model.Item
3
4 val sampleItems = listOf(
5     Item("Indonesia Open", "Indonesia Open adalah
6     sebuah kejuaraan bulu tangkis internasional di Indonesia.
7     Kejuaraan ini diselenggarakan oleh Persatuan Bulu Tangkis
8     Seluruh Indonesia dengan edisi terkini berlangsung di Istora
9     Gelora
10    Bung Karno, Jakarta.", "https://img.bwfbadminton.com/image/u
11    pload/v1746_607228/assets/tournaments/logo/8E66BAA2-CF0F
12    -4FFA-8798-FF700D1CDBEC.png", "https://bwfworldtour.bwfb
13    adminton.com/tournament/5236/indonesia-open-2025/results/2
14    025-06-03"), Item("Yonex All England Open", "Turnamen
15    bulutangkis tertua di dunia yang diadakan di Inggris sejak
16    1899. Saat ini menjadi bagian dari BWF World Tour Super 1000
17    dan sangat bergengsi karena Sejarah dan
18    prestisenya.", "https://img.bwfbadminton.com/image/upload
19    /v1734076435/assets/tournaments/logo/F6304F7C-D298-4E3D
20    AA62A6B933CBB8C7.png", "https://bwfworldtour.bwfbadminton.com/
21    tournament/5238/yonex-all-england-open-badminton-champions
22    hips-2025/results/podium/"),
23    Item("Victor China Open", "Turnamen bulutangkisintern
24    asional bergengsi yang diadakan setiap tahun di Tiongkok.
25    Merupakan bagian dari BWF World Tour Super 1000 (bersama
26    All England dan Indonesia Open). Menarik banyak pemain top
27    dunia karena hadiah besar dan poin peringkat
28    tinggi.", "https://img.bwfbadminton.com/image/upload/v17
29    46149516/assets/tournaments/logo/5A59A1E7-1366-49BA-863D

```

```

30 -33FB844D51C0.png", "https://bwfworldtour.bwfbadmin
31 ton.com/tournament/5281/victor-china-open-2025/results
32 /2025-07-22"),
33 Item("Daihatsu Indonesia Master", "Indonesia Mast
34 er adalah sebuah kejuaraan bulu tangkis internasional di
35 Indonesia. Kejuaraan ini diselenggarakan oleh Persatuan Bulu
36 Tangkis Seluruh Indonesia dengan edisi terkini berlangsung di
37 Istora Gelora Bung Karno,
38 Jakarta.", "https://img.bwfbadminton.com/image/upload/
39 v1731292348/assets/tournaments/logo/C166BF11-2128-4F8C-B1ED-
40 F30565DE9089.png",
41 "https://bwfworldtour.bwfbadminton.com/tournament/5234/
42 daihatsu-indonesia-masters-2025/results/podium/"),
43 Item("HSBC BWF World Tour Final", "Turnamen penutup
44 musim bulutangkis yang hanya mempertemukan 8 pemain atau
45 pasangan terbaik dunia dari setiap sektor berdasarkan
46 perolehan poin sepanjang musim BWF World Tour. Turnamen ini
47 sangat bergengsi dan berhadiah besar, setara dengan kejuaraan
48 elit lainnya.", "https://img.bwfbadminton.com/image/
49 upload/v1734514049/assets/tournaments/logo/F34EBDDDB-AA5D-
50 4F1E-A14F-
51 A94473DD1295.png", "https://bwfworldtourfinals.bwfbadm
52 inton.com/results/5259/hsbc-bwf-world-tour-finals-
53 2025/draw/"),
54 Item("TotalEnergies BWF Sudirman Cup Finals", "Sudirman Cup
55 adalah kejuaraan bulu tangkis internasional untuk nomor beregu
56 campuran, mempertandingkan nomor tunggal putra, tunggal putri,
57 ganda putra, ganda putri, dan ganda campuran.
58 Kejuaraan ini digelar setiap dua tahun
59 sekali.", "https://img.bwfbadminton.com/image/upload/v
60 1734511970/assets/tournaments/logo/C9F72B3B-2C6A-4D63-
61 A782-78968933BDE9.png", "https://bwfsudirmancup.bwfb
62 adminton.com/results/5260/totalenergies-bwf-sudirman-cu
63 p-finals-2025/podium"),
64 ListItem("TotalEnergies BWF Thomas & Uber Cup Finals",
65 "Thomas Cup dan Uber Cup adalah kejuaraan bulutangkis
66 beregu putra dan putri antarnegara yang diselenggarakan
67 dua tahun sekali oleh BWF. Setiap tim terdiri dari pemain
68 tunggal dan ganda, dan bertanding untuk menjadi tim
69 nasional putra dan putri terbaik di dunia",
70 "https://bwfthomasubercups.bwfbadminton.com/wp-conten
71 t/plugins/bwf-menu
72 system/images/TUCTournamentLogoLandscape.png",
73 "https://bwfthomasubercups.bwfbadminton.com/results/5
74 147/totalenergies-bwf-thomas-uber-cup-finals-2024/podium"),
75 )

```

Item.kt

Table 12. Source Code Item.kt

1	package com.example.modul4.model
2	
3	data class Item(4 val id: Int, 5 val title: String, 6 val desc: String, 7 val imageUrl: String, 8 val linkUrl: String)

NavGraph.kt

Table 13. Source Code NavGraph.kt

1	package com.example.modul4.navigation
2	
3	import androidx.compose.runtime.Composable
4	import androidx.compose.ui.Modifier
5	import androidx.lifecycle.viewmodel.compose.viewModel
6	import androidx.navigation.NavType
7	import androidx.navigation.compose.NavHost
8	import androidx.navigation.compose.composable
9	import androidx.navigation.navArgument
10	import androidx.navigation.NavHostController
11	import com.example.modul4.screen.DetailScreen
12	import com.example.modul4.screen.ListScreen
13	import com.example.modul4.viewModel.ItemViewModel
14	import com.example.modul4.viewModel.ItemViewModelFactory
15	
16	@Composable
17	fun AppNavGraph(18 navController: NavHostController, 19 modifier: Modifier = Modifier 20) { 21 val factory = ItemViewModelFactory() 22 val itemViewModel: ItemViewModel = viewModel(factory 23 = factory) 24 25 NavHost(navController = navController, 26 startDestination = "list", modifier = modifier) { 27 composable("list") { 28 ListScreen(navController = navController, 29 viewModel = itemViewModel) 30 } 31 composable(

32	"detail/{id}",	
33	arguments = listOf(navArgument("id") {	
34	type = NavType.IntType	
35	})	
36	) { backStackEntry ->	
37	val id	=
38	backStackEntry.arguments?.getInt("id") ?: 0	
39	DetailScreen(id = id, viewModel	=
40	itemViewModel)	
41	}	
41	}	
42	}	

DetailScreen.kt

Table 14. Source Code DetailScreen.kt

1	package com.example.modul4.screen	
2		
3	import androidx.compose.foundation.layout.*	
4	import	
5	androidx.compose.foundation.shape.RoundedCornerShape	
6	import androidx.compose.material3.*	
7	import androidx.compose.runtime.Composable	
8	import androidx.compose.ui.Alignment	
9	import androidx.compose.ui.Modifier	
10	import androidx.compose.ui.draw.clip	
11	import androidx.compose.ui.layout.ContentScale	
12	import androidx.compose.ui.text.font.FontWeight	
13	import androidx.compose.ui.unit.dp	
14	import coil.compose.AsyncImage	
15	import com.example.modul4.viewModel.ItemViewModel	
16		
17	@Composable	
18	fun DetailScreen(viewModel: ItemViewModel, id: Int) {	
19	val item = viewModel.getItemById(id)	
20		
21	if (item == null) {	
22	// Item tidak ditemukan	
23	Box(	
24	modifier = Modifier.fillMaxSize(),	
25	contentAlignment = Alignment.Center	
26	) {	
27	Text(	
28	text = "Item tidak ditemukan",	
29	style	=
30	MaterialTheme.typography.bodyLarge.copy(	

31	fontWeight = FontWeight.Bold,	
32	color	=
33	MaterialTheme.colorScheme.error	
34	)	
35	)	
36	}	
37	} else {	
38	Column(	
39	horizontalAlignment	=
40	Alignment.CenterHorizontally,	
41	modifier = Modifier.padding(16.dp)	
42	) {	
43	Card(	
44	shape = RoundedCornerShape(16.dp),	
45	modifier = Modifier	
46	.fillMaxWidth()	
47	.height(250.dp)	
48	.clip(RoundedCornerShape(16.dp))	
49	) {	
50	AsyncImage(	
51	model = item.imageUrl,	
52	contentDescription = item.title,	
53	modifier = Modifier	
54	.fillMaxSize()	
55		
56	.clip(RoundedCornerShape(16.dp)),	
57	contentScale = ContentScale.Crop	
58	)	
59	}	
60		
61	Spacer(modifier = Modifier.height(24.dp))	
62		
63	Text(	
64	text = item.title,	
65	style	=
66	MaterialTheme.typography.titleMedium.copy(	
67	fontWeight = FontWeight.Bold,	
68	color	=
69	MaterialTheme.colorScheme.primary	
70	),	
71	modifier = Modifier	
72	.fillMaxWidth()	
73	.padding(horizontal = 16.dp)	
74	.align(Alignment.CenterHorizontally)	
75	)	
76		
77	Spacer(modifier = Modifier.height(8.dp))	

78	
79	Text(
80	text = "Description:",
81	style
82	MaterialTheme.typography.bodyMedium.copy(
83	fontWeight = FontWeight.SemiBold,
84	color
85	MaterialTheme.colorScheme.secondary
86	),
87	modifier = Modifier.padding(horizontal =
88	16.dp)
89	)
90	
91	Text(
92	text = item.desc,
93	style
94	MaterialTheme.typography.bodyLarge,
95	modifier = Modifier.padding(horizontal =
96	16.dp)
97	)
98	}
99	}
100	}
101	

ListScreen.kt

Table 15. Source Code ListScreen.kt

1	package com.example.modul4.screen
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.*
7	import
8	androidx.compose.foundation.shape.RoundedCornerShape
9	import androidx.compose.foundation.clickable
10	import androidx.compose.foundation.lazy.LazyColumn
11	import androidx.compose.foundation.lazy.items
12	import androidx.compose.material3.*
13	import androidx.compose.runtime.Composable
14	import androidx.compose.ui.Modifier
15	import androidx.compose.ui.draw.clip
16	import androidx.compose.ui.layout.ContentScale
17	import androidx.compose.ui.platform.LocalContext
18	import androidx.compose.ui.text.font.FontWeight


```

19 import androidx.compose.ui.unit.dp
20 import androidx.core.content.ContextCompat
21 import androidx.navigation.NavController
22 import coil.compose.rememberAsyncImagePainter
23 import com.example.modul4.data.ListItem
24 import com.example.modul4.navigation.Screen
25
26 @Composable
27 fun HomeScreen(navController: NavController, items:
28 List<ListItem>) {
29     LazyColumn(
30         modifier = Modifier
31             .fillMaxSize()
32             .padding(8.dp)
33     ) {
34         items(items) { item ->
35             Card(
36                 modifier = Modifier
37                     .padding(8.dp)
38                     .fillMaxWidth(),
39                 shape = RoundedCornerShape(16.dp),
40                 elevation =
41 CardDefaults.cardElevation(4.dp)
42             ) {
43                 Column(
44                     modifier = Modifier.padding(16.dp)
45                 ) {
46                     Image(
47                         painter =
48 rememberAsyncImagePainter(item.imageUrl),
49                         contentDescription = null,
50                         modifier = Modifier
51                             .fillMaxWidth()
52                             .height(180.dp)
53
54                     .clip(RoundedCornerShape(16.dp)),
55                         contentScale = ContentScale.Crop
56                     )
57                     Spacer(modifier =
58 Modifier.height(8.dp))
59                     Text(item.title,
60                         fontWeight =
61 FontWeight.Bold)
62                     Spacer(modifier =
63 Modifier.height(8.dp))
64
65                     val context = LocalContext.current

```

66	
67	Row(
68	modifier
69	Modifier.fillMaxWidth(),
70	horizontalArrangement
71	Arrangement.SpaceBetween
72	) {
73	Button(onClick = {
74	val intent
75	Intent(Intent.ACTION_VIEW, Uri.parse(item.url))
76	
77	intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
78	
79	ContextCompat.startActivity(context, intent, null)
80	}) {
81	Text("Open Link")
82	}
83	Button(onClick = {
84	
85	navController.navigate(Screen.Detail.createRoute(item.id))
86	}) {
87	Text("Detail")
88	}
89	}
90	}
91	}
92	}
93	}
94	}

ItemViewModel.kt

Table 16. Source Code ItemViewModel.kt

1	package com.example.modul4.viewModel
2	
3	import androidx.lifecycle.ViewModel
4	import kotlinx.coroutines.flow.MutableStateFlow
5	import kotlinx.coroutines.flow.StateFlow
6	import com.example.modul4.model.Item
7	import com.example.modul4.data.sampleItems
8	import android.util.Log
9	
10	class ItemViewModel : ViewModel() {
11	private val _itemList
12	MutableStateFlow<List<Item>>(emptyList())
13	val itemList: StateFlow<List<Item>> = _itemList

14	
15	private val _selectedItem =
16	MutableStateFlow<Item?>(null)
17	val selectedItem: StateFlow<Item?> get() =
18	_selectedItem
19	
20	init {
21	_itemList.value = sampleItems
22	Log.d("ItemViewModel", "Item list initialized:
23	\${sampleItems.map { it.title }}")
24	}
25	
26	fun selectItem(item: Item) {
27	_selectedItem.value = item
28	Log.d("ItemViewModel", "Selected item: \$item")
29	}
30	
31	fun getItemById(id: Int): Item? {
32	return _itemList.value.find { it.id == id }
33	}
34	}
35	

ItemViewModelFactory.kt

Table 17. Source Code ItemViewModelFactory.kt

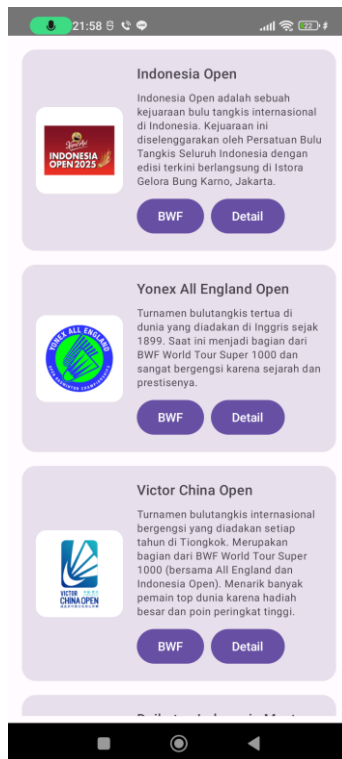
1	package com.example.modul4.viewModel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class ItemViewModelFactory : ViewModelProvider.Factory {
7	override fun <T : ViewModel> create(modelClass:
8	Class<T>): T {
9	if
10	(modelClass.isAssignableFrom(ItemViewModel::class.java))
11	{
12	return ItemViewModel() as T
13	}
14	throw IllegalArgumentException("Unknown ViewModel
15	class")
16	}
17	}

MainActivity.kt

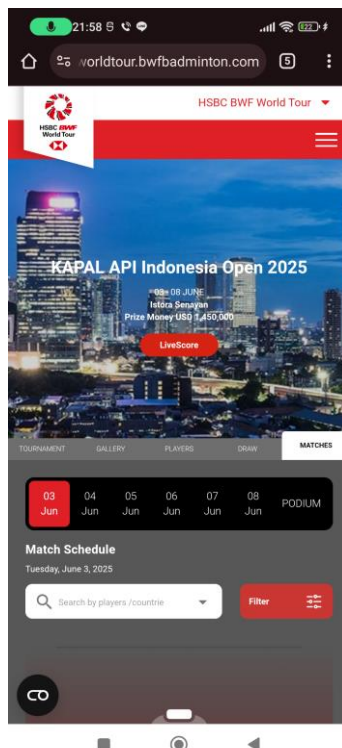
Table 18. Source Code MainActivity.kt

1	package com.example.modul4
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.material3.MaterialTheme
7	import androidx.compose.material3.Surface
8	import androidx.navigation.compose.rememberNavController
9	import com.example.modul4.navigation.AppNavGraph
10	
11	class MainActivity : ComponentActivity() {
12	override fun onCreate(savedInstanceState: Bundle?) {
13	super.onCreate(savedInstanceState)
14	setContent {
15	Surface(color
16	MaterialTheme.colorScheme.background) {
17	val
18	rememberNavController()
19	AppNavGraph(navController
20	navController)
21	}
22	}
23	}
24	}

B. Output Program



Gambar 11. Screenshot Halaman Awal



Gambar 12. Screenshot Ketika Tombol Ditekan Mengarah Ke Website



Gambar 13. Screenshot Ketika Tombol Detail Ditekan

C. Pembahasan

MainActivity.kt:

Pada line 1, dideklarasikan nama package file Kotlin, yaitu `com.example.modul4`.

Pada line 3 sampai 7, dilakukan import terhadap class dan fungsi yang dibutuhkan seperti `ComponentActivity`, `setContent`, komponen material 3, dan navigasi `Compose`.

Pada line 9, dideklarasikan class `MainActivity` yang merupakan subclass dari `ComponentActivity`. Class ini menjadi titik masuk utama dari aplikasi Android.

Pada line 10, dideklarasikan fungsi `onCreate` yang merupakan override dari fungsi lifecycle activity Android, dan menerima parameter `savedInstanceState` bertipe `Bundle?`.

Pada line 11, dipanggil `super.onCreate(savedInstanceState)` untuk menjalankan logika bawaan dari `ComponentActivity`.

Pada line 12, dipanggil `setContent` untuk menentukan UI dengan menggunakan Jetpack Compose.

Pada line 13, dibuat Surface dengan warna latar belakang sesuai dengan tema aplikasi (`MaterialTheme.colorScheme.background`).

Pada line 14, dideklarasikan `NavController` menggunakan `rememberNavController()` yang digunakan untuk mengelola navigasi antar layar.

Pada line 15, dipanggil fungsi `AppNavGraph` dan diberikan parameter `NavController` untuk mengatur jalur navigasi aplikasi.

Pada line 16, penutupan Surface.

Pada line 17, penutupan `setContent`.

Pada line 18, penutupan fungsi `onCreate`.

Pada line 19, penutupan class `MainActivity`.

DataTournament.kt

Pada line 1, dideklarasikan nama package file Kotlin, yaitu `com.example.modul4.data`.

Pada line 2, dilakukan import terhadap `Item` dari package `com.example.modul4.model` agar bisa digunakan dalam daftar data.

Pada line 4, dideklarasikan variabel `sampleItems` sebagai list yang berisi beberapa objek `Item`. List ini merupakan data contoh yang akan ditampilkan di aplikasi.

Pada line 5, dibuat objek `Item` pertama dengan id 1, judul "Indonesia Open", deskripsi mengenai turnamen, URL logo turnamen, dan URL resmi hasil turnamen.

Pada line 6, dibuat objek `Item` kedua dengan id 2, berjudul "Yonex All England Open", dengan deskripsi tentang sejarah turnamen tertua di dunia, URL logo, dan URL hasil turnamen.

Pada line 7, dibuat objek `Item` ketiga dengan id 3, berjudul "Victor China Open", beserta deskripsi turnamen, logo, dan tautan resmi hasil pertandingan.

Pada line 8, dibuat objek `Item` keempat dengan id 4, berjudul "Daihatsu Indonesia Master", dengan informasi deskripsi, logo turnamen, dan tautan hasil pertandingan.

Pada line 9, dibuat objek Item kelima dengan id 5, berjudul "HSBC BWF World Tour Final", dengan deskripsi tentang turnamen final musim, logo turnamen, dan URL hasil turnamen.

Pada line 10, dibuat objek Item keenam dengan id 6, berjudul "TotalEnergies BWF Sudirman Cup Finals", dengan deskripsi tentang kejuaraan beregu campuran, logo, dan tautan hasil.

Pada line 11, dibuat objek Item ketujuh dengan id 7, berjudul "TotalEnergies BWF Thomas & Uber Cup Finals", dengan deskripsi mengenai kejuaraan beregu putra dan putri, logo, dan URL hasil pertandingan.

Pada line 12, penutupan dari deklarasi list sampleItems.

Item.kt

Pada line 1, dideklarasikan nama package file Kotlin, yaitu com.example.modul4.model.

Pada line 3, didefinisikan sebuah data class dengan nama Item. Data class digunakan untuk merepresentasikan data dan secara otomatis menyediakan fungsi seperti toString(), equals(), dan hashCode().

Pada line 4, dideklarasikan properti id bertipe Int yang berfungsi sebagai identifikasi unik setiap item.

Pada line 5, dideklarasikan properti title bertipe String yang berfungsi sebagai judul atau nama dari item.

Pada line 6, dideklarasikan properti desc bertipe String yang berisi deskripsi atau penjelasan tentang item.

Pada line 7, dideklarasikan properti imageUrl bertipe String yang menyimpan URL dari gambar atau logo item.

Pada line 8, dideklarasikan properti linkUrl bertipe String yang berisi tautan menuju halaman detail atau hasil turnamen.

Pada line 9, dilakukan penutupan deklarasi class Item.

NavGraph.kt

Pada line 1, dideklarasikan nama package file Kotlin, yaitu `com.example.modul4.navigation`.

Pada line 3–9, dilakukan import terhadap berbagai komponen yang dibutuhkan untuk membangun navigasi dan tampilan menggunakan Jetpack Compose, seperti `Composable`, `NavHost`, `composable`, dan `NavHostController`.

Pada line 10–13, dilakukan import terhadap layar/tampilan `DetailScreen` dan `ListScreen`, serta `ItemViewModel` dan `ItemViewModelFactory` yang akan digunakan untuk manajemen data.

Pada line 15, didefinisikan fungsi `AppNavGraph` sebagai fungsi `@Composable` yang menerima parameter `navController` bertipe `NavHostController` dan modifier opsional bertipe `Modifier`.

Pada line 17, dibuat instance `ItemViewModelFactory` yang akan digunakan untuk menghasilkan `ItemViewModel`.

Pada line 18, dibuat instance `ItemViewModel` menggunakan fungsi `viewModel()` dengan factory yang telah dibuat.

Pada line 20, dideklarasikan `NavHost` yang menjadi container untuk semua destinasi navigasi. Start destination-nya adalah `"list"`, dan modifier diteruskan dari parameter fungsi.

Pada line 21–23, didefinisikan destinasi `"list"` yang akan memanggil fungsi `ListScreen` dan mengoper `navController` serta `itemViewModel`.

Pada line 24–31, didefinisikan destinasi `"detail/{id}"` yang menerima argumen `id` bertipe `Int`. Nilai `id` diambil dari `backStackEntry` dan diteruskan ke `DetailScreen` bersama dengan `itemViewModel`.

Pada line 32, dilakukan penutupan dari blok `NavHost`.

Pada line 33, dilakukan penutupan dari fungsi `AppNavGraph`.

DetailScreen.kt

Pada line 1, dideklarasikan nama package file Kotlin, yaitu `com.example.modul4.screen`.

Pada line 3–13, dilakukan import terhadap berbagai komponen Jetpack Compose yang digunakan dalam pembuatan UI, seperti layout (Column, Box, Spacer), elemen visual (Text, Card), dan style (RoundedCornerShape, FontWeight, dll).

Pada line 14, dilakukan import terhadap AsyncImage dari library Coil untuk menampilkan gambar dari URL.

Pada line 15, dilakukan import terhadap ItemViewModel dari package com.example.modul4.viewModel yang digunakan untuk mendapatkan data item.

Pada line 17, didefinisikan fungsi DetailScreen sebagai fungsi @Composable dengan parameter viewModel dan id.

Pada line 18, data item diambil dari viewModel menggunakan fungsi getItemById(id).

Pada line 20–28, dilakukan pengecekan apakah item ditemukan atau tidak. Jika item == null, maka akan ditampilkan teks "Item tidak ditemukan" di tengah layar dengan gaya teks berwarna merah dan tebal.

Pada line 29, jika item ditemukan, UI akan ditampilkan dalam bentuk kolom (Column) dengan posisi horizontal di tengah dan padding sebesar 16dp.

Pada line 30–37, sebuah Card dibuat untuk menampilkan gambar dari item dengan bentuk rounded corner dan tinggi 250dp. Di dalamnya, AsyncImage digunakan untuk menampilkan gambar berdasarkan item.imageUrl. Gambar akan diisi penuh (fillMaxSize()) dan dipotong agar sesuai dengan bentuk rounded.

Pada line 39, Spacer digunakan untuk memberikan jarak vertikal sebesar 24dp.

Pada line 41–47, ditampilkan judul item (item.title) dengan gaya teks tebal dan warna utama aplikasi. Teks memenuhi lebar penuh dan ditempatkan di tengah.

Pada line 49–54, label "Description:" ditampilkan sebelum deskripsi item, menggunakan gaya teks semi-bold dan warna sekunder.

Pada line 56–58, deskripsi lengkap dari item (item.desc) ditampilkan dalam gaya teks body.

Pada line 59, penutupan dari blok fungsi DetailScreen.

ListScreen.kt

Pada line 1, dideklarasikan package `com.example.modul4.screen`. Pada line 3–5, dilakukan import terhadap `Intent`, `Uri`, dan `Log` yang digunakan untuk eksplisit `intent` dan `logging`. Pada line 6–31, berbagai komponen dari Jetpack Compose diimpor, seperti `layout` (`Row`, `Column`, `Spacer`), gambar (`AsyncImage`), tema (`MaterialTheme`), dan lain-lain. Pada line 32, `LocalContext` diimpor untuk mendapatkan context saat ini. Pada line 33, `viewModel` digunakan dari `lifecycle library` untuk menghubungkan ke `ItemViewModel`.

Pada line 34, `NavController` diimpor dari `Navigation Compose`. Pada line 35, `AsyncImage` dari `Coil` digunakan untuk memuat gambar dari URL. Pada line 36, `ItemViewModel` diimpor dari package `viewModel`.

Pada line 38, didefinisikan fungsi `@Composable` bernama `ListScreen`, dengan parameter `navController` dan `viewModel`. Pada line 39, `itemList` diambil dari `viewModel.itemList` menggunakan `collectAsState()`.

Pada line 40, context didapatkan dari `LocalContext.current`, digunakan untuk membuat intent eksplisit.

Pada line 42, digunakan `LazyColumn` untuk menampilkan daftar item secara efisien. Pada line 45, digunakan `items()` untuk menampilkan setiap item dalam `itemList`.

Pada line 46–52, sebuah `Card` digunakan untuk menampilkan setiap item, dengan bentuk rounded dan padding. Komponen ini juga clickable, yang akan memanggil `selectItem()` di `viewModel` dan menavigasi ke screen detail berdasarkan `item.id`.

Pada line 53–60, digunakan `Row` untuk menyusun gambar dan teks secara horizontal. Pada line 61–66, `AsyncImage` menampilkan gambar dengan ukuran 100dp dan rounded corner.

Pada line 68, `Spacer` digunakan untuk memberikan jarak horizontal.

Pada line 70–74, `Column` digunakan untuk menampilkan judul dan deskripsi item secara vertikal.

Pada line 75, `Row` digunakan untuk menampung dua Button: satu untuk membuka link, satu lagi untuk menavigasi ke detail.

Pada line 76–81, Button pertama menjalankan explicit intent (Intent.ACTION_VIEW) ke item.linkUrl, membuka browser atau aplikasi lain. Pada line 83–87, Button kedua menjalankan navigasi ke detail screen ("detail/\${item.id}") secara langsung dari tombol "Detail". Pada line 89, fungsi ListScreen ditutup.

ItemViewModel.kt

Pada line 1, dideklarasikan package com.example.modul4.viewModel.

Pada line 3 sampai 8, dilakukan import terhadap beberapa library dan class yang dibutuhkan, yaitu ViewModel dari Android Jetpack Lifecycle untuk manajemen siklus hidup data, MutableStateFlow dan StateFlow dari Kotlin Coroutines yang digunakan untuk mengelola data secara reaktif dan dapat diobservasi, class Item dari package model yang merepresentasikan data item, sampleItems sebagai data contoh dari package data, serta Log untuk pencatatan log debug.

Pada line 10, dideklarasikan class ItemViewModel yang merupakan turunan dari ViewModel.

Pada line 11, dibuat variabel privat _itemList bertipe MutableStateFlow yang menyimpan daftar item dengan nilai awal berupa list kosong.

Pada line 12, dideklarasikan variabel publik itemList bertipe StateFlow yang hanya bisa dibaca dan mengambil nilai dari _itemList, sehingga data item dapat dipantau dari luar class tanpa bisa diubah langsung.

Pada line 14, dibuat variabel privat _selectedItem bertipe MutableStateFlow yang menyimpan item yang sedang dipilih, dengan nilai awal null.

Pada line 15, variabel publik selectedItem bertipe StateFlow disediakan sebagai getter untuk mengakses nilai _selectedItem secara read-only.

Pada line 17 sampai 20, terdapat blok init yang merupakan inisialisasi ketika ItemViewModel dibuat. Pada blok ini, nilai _itemList diisi dengan data contoh sampleItems. Selain itu, sebuah log debug dicatat yang menampilkan daftar judul item yang sudah diinisialisasi.

Pada line 22 sampai 25, terdapat fungsi `selectItem` yang menerima parameter `item` bertipe `Item`. Fungsi ini mengubah nilai `_selectedItem` menjadi `item` yang dipilih dan mencatat log debug terkait `item` tersebut.

Pada line 27 sampai 29, terdapat fungsi `getItemById` yang menerima parameter `id` bertipe `Int`. Fungsi ini mencari dan mengembalikan objek `Item` dari `_itemList` yang memiliki `id` sesuai parameter. Jika tidak ditemukan, fungsi mengembalikan `null`.

ItemViewModelFactory.kt

Pada line 1, dideklarasikan package `com.example.modul4.viewModel`.

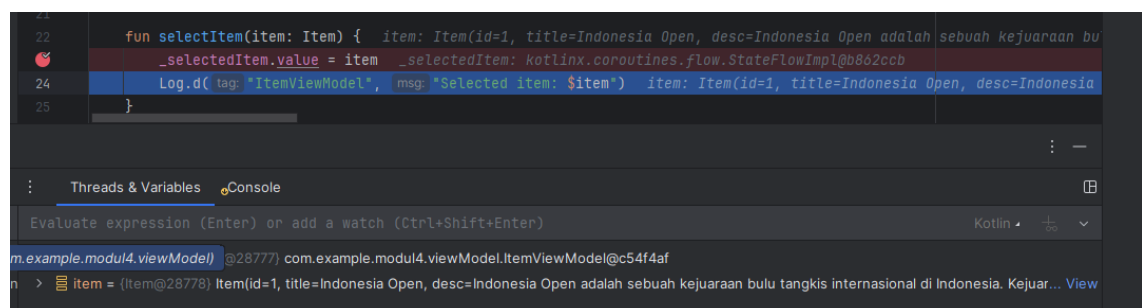
Pada line 3 dan 4, dilakukan import terhadap `ViewModel` dan `ViewModelProvider` dari Android Jetpack Lifecycle yang digunakan untuk membuat dan menyediakan instance `ViewModel` dengan siklus hidup yang benar.

Pada line 6 sampai 13, dideklarasikan class `ItemViewModelFactory` yang mengimplementasikan interface `ViewModelProvider.Factory`.

Pada line 7 sampai 11, terdapat override fungsi `create` yang bertugas membuat instance `ViewModel` berdasarkan kelas yang diminta. Fungsi ini menerima parameter `modelClass` bertipe `Class<T>`, kemudian memeriksa apakah kelas tersebut merupakan subclass dari `ItemViewModel`. Jika ya, maka dibuat dan dikembalikan instance `ItemViewModel` yang di-cast ke tipe `T`. Jika bukan, maka fungsi melemparkan `IllegalArgumentException` dengan pesan "Unknown ViewModel class" sebagai tanda bahwa kelas `ViewModel` yang diminta tidak dikenali atau tidak didukung oleh factory ini.

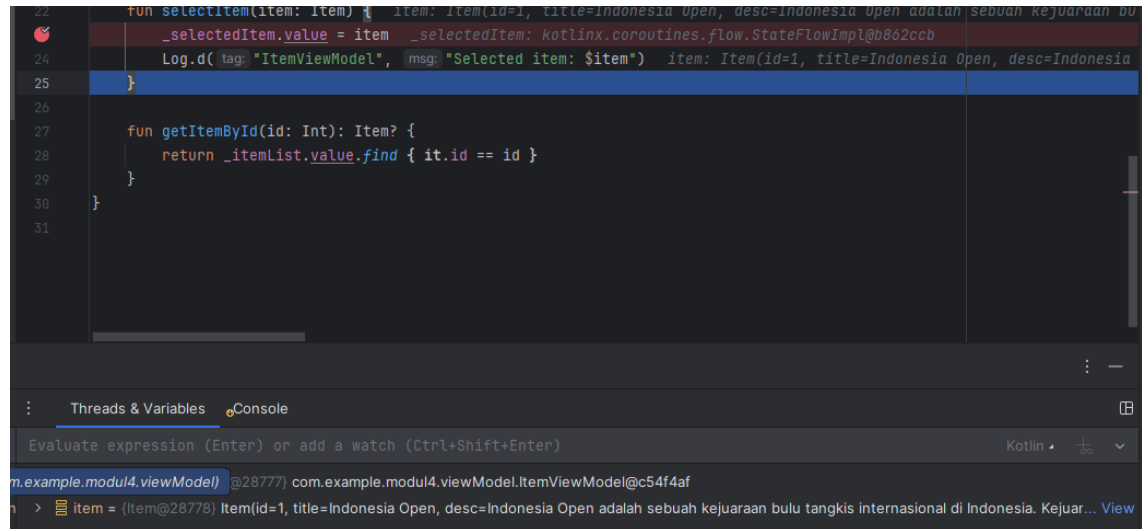
Debugging

1. Step Into



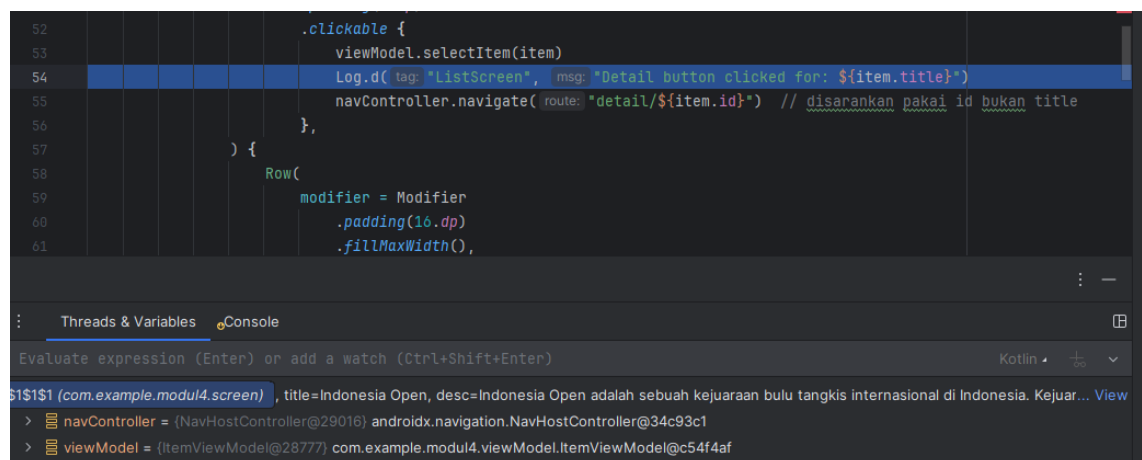
Ketika menekan Step Into di baris `_selectedItem.value = item`, debugger akan masuk ke implementasi properti `value` dari `MutableStateFlow`. Karena ini adalah bagian dari library Kotlin Coroutine, maka akan terlihat kode internal yang mengatur nilai baru disimpan dan bagaimana observer yang melihat perubahan nilai diberi tahu.

2. Step Over



Ketika melakukan Step Over, maka debugger akan mengeksekusi seluruh instruksi `_selectedItem.value = item` secara utuh tanpa masuk ke implementasi internalnya. Ini berguna jika tidak ingin melihat detail internal `MutableStateFlow`, dan langsung lanjut ke baris kode berikutnya.

3. Step Out



Jika sudah masuk ke dalam fungsi internal atau method dan ingin keluar kembali ke kode pemanggil, maka bisa menggunakan Step Out. Step Out akan mengeksekusi sisa

kode di dalam fungsi tersebut dan mengembalikan kontrol ke baris setelah pemanggilan fungsi yang kamu masuki sebelumnya.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Jawaban

Application class dalam arsitektur aplikasi Android adalah kelas dasar yang dijalankan terlebih dahulu saat aplikasi mulai berjalan. Kelas ini merupakan subclass dari `android.app.Application` dan berfungsi sebagai titik awal untuk menginisialisasi komponen atau data global yang diperlukan oleh seluruh aplikasi selama siklus hidupnya. Fungsi utama Application class adalah menyediakan konteks aplikasi yang bersifat global dan bertahan selama aplikasi aktif. Dengan Application class, developer dapat melakukan inisialisasi satu kali seperti setup library pihak ketiga, konfigurasi database, atau penyimpanan data yang harus tersedia secara konsisten di seluruh bagian aplikasi. Selain itu, Application class membantu mengelola state aplikasi secara keseluruhan dan dapat digunakan untuk menyimpan data atau variabel yang perlu diakses oleh banyak komponen aplikasi tanpa perlu membuat ulang atau melewatkan data melalui Intent atau Bundle.

MODUL 5 : Connect to the Internet

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- Gunakan caching strategy pada Room..
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

MainActivity.kt

Table 19. Source Code MainActivity.kt

1	package com.example.modul5
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.widget.Toast
7	import androidx.activity.ComponentActivity
8	import androidx.activity.compose.setContent
9	import androidx.compose.foundation.Image
10	import androidx.compose.foundation.layout.*

```

11 import androidx.compose.foundation.lazy.LazyColumn
12 import androidx.compose.foundation.lazy.items
13 import androidx.compose.material3.*
14 import androidx.compose.runtime.*
15 import androidx.compose.ui.Alignment
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.platform.LocalContext
18 import androidx.compose.ui.res.painterResource
19 import androidx.compose.ui.unit.dp
20 import androidx.lifecycle.ViewModel
21 import androidx.lifecycle.ViewModelProvider
22 import androidx.lifecycle.viewmodel.compose.viewModel
23 import androidx.navigation.NavHostController
24 import
25 androidx.navigation.compose.rememberNavController
26 import com.example.modul5.data.api.RetrofitInstance
27 import com.example.modul5.data.local.PostDatabase
28 import com.example.modul5.data.models.Post
29 import
30 com.example.modul5.data.repository.PostRepository
31 import com.example.modul5.viewModel.PostViewModel
32
33 class MainActivity : ComponentActivity() {
34
35     class PostViewModelFactory(
36         private val repository: PostRepository
37     ) : ViewModelProvider.Factory {
38         override fun <T : ViewModel> create(modelClass:
39 Class<T>): T {
40             if
41 (modelClass.isAssignableFrom(PostViewModel::class.java
42 )) {
43                 @Suppress("UNCHECKED_CAST")
44                 return PostViewModel(repository) as T
45             }
46             throw      IllegalArgumentException("Unknown
47 ViewModel class")
48         }
49     }
50
51     @OptIn(ExperimentalMaterial3Api::class)
52     override fun onCreate(savedInstanceState: Bundle?)
53 {
54         super.onCreate(savedInstanceState)
55
56         val api = RetrofitInstance.api
57

```

```

58         val dao =
59         PostDatabase.getDatabase(applicationContext).postDao()
60         val repository = PostRepository(api, dao)
61
62         val factory = PostViewModelFactory(repository)
63
64         setContent {
65             val postViewModel: PostViewModel =
66             viewModel(factory = factory)
67
68             val posts by
69             postViewModel.posts.collectAsState()
70             val favoritePosts by
71             postViewModel.favorites.collectAsState()
72             val errorMessage by
73             postViewModel.errorMessage.collectAsState()
74
75             val navController = rememberNavController()
76             var selectedTab by remember {
77             mutableStateOf("list") }
78
79             Scaffold(
80                 topBar = {
81                     TopAppBar(title = { Text("Daftar
82                     Doa") })
83                 },
84                 bottomBar = {
85                     NavigationBar {
86                         NavigationBarItem(
87                             selected = selectedTab ==
88                             "list",
89                             onClick = { selectedTab =
90                             "list" },
91                             icon = {
92                             Icon(painterResource(id =
93                             android.R.drawable.ic_menu_agenda),
94                             contentDescription = "List") },
95                             label = { Text("List") }
96                         )
97                         NavigationBarItem(
98                             selected = selectedTab ==
99                             "favorit",
100                             onClick = { selectedTab =
101                             "favorit" },
102                             icon = {
103                             Icon(painterResource(id = android.R.drawable.star_on),
104                             contentDescription = "Favorit") },

```

```

105         label = { Text("Favorit") }
106     )
107     }
108 }
109 ) { paddingValues ->
110
111     val context = LocalContext.current
112
113     val onAddFavoriteWithToast: (Post) ->
114 Unit = { post ->
115         postViewModel.addToFavorite(post)
116         Toast.makeText(context, "Added to
117 favorites", Toast.LENGTH_SHORT).show()
118     }
119     val onRemoveFavoriteWithToast: (Post) -
120 > Unit = { post ->
121
122     postViewModel.removeFromFavorite(post)
123         Toast.makeText(context, "Removed
124 from favorites", Toast.LENGTH_SHORT).show()
125     }
126
127     when (selectedTab) {
128         "list" -> {
129             if
130 (!errorMessage.isNullOrEmpty()) {
131                 Text(
132                     text = errorMessage ?:
133                     "",
134                     modifier = Modifier
135                         .padding(16.dp)
136
137                     .padding(paddingValues),
138                     color
139                     =
140                     MaterialTheme.colorScheme.error
141                 )
142             } else {
143                 DoaList(
144                     posts = posts,
145                     navController
146                     =
147                     navController,
148                     onAddFavorite
149                     =
150                     onAddFavoriteWithToast,
151                     modifier
152                     =
153                     Modifier.padding(paddingValues)
154                 )
155             }
156         }
157     }
158 }
159 }

```

```

152         }
153         "favorit" -> {
154             FavoritScreen(
155                 favorites = favoritePosts,
156                 onRemoveFavorite =
157 onRemoveFavoriteWithToast,
158                 modifier =
159 Modifier.padding(paddingValues)
160             )
161         }
162     }
163 }
164 }
165 }
166 }
167
168 @Composable
169 fun DoaList(
170     posts: List<Post>,
171     navController: NavHostController,
172     onAddFavorite: (Post) -> Unit,
173     modifier: Modifier = Modifier
174 ) {
175     LazyColumn(
176         contentPadding = PaddingValues(16.dp),
177         verticalArrangement =
178 Arrangement.spacedBy(12.dp),
179         modifier = modifier
180     ) {
181         items(posts) { post ->
182             DoaItem(
183                 post = post,
184                 navController = navController,
185                 onAddFavorite = onAddFavorite
186             )
187         }
188     }
189 }
190
191 @Composable
192 fun FavoritScreen(
193     favorites: List<Post>,
194     onRemoveFavorite: (Post) -> Unit,
195     modifier: Modifier = Modifier
196 ) {
197     if (favorites.isEmpty()) {
198

```

```

199         Box(modifier = modifier.fillMaxSize(),
200 contentAlignment = Alignment.Center) {
201             Text(text = "Halaman Favorit masih kosong",
202 style = MaterialTheme.typography.titleMedium)
203         }
204     } else {
205         LazyColumn(
206             contentPadding = PaddingValues(16.dp),
207             verticalArrangement =
208 Arrangement.spacedBy(12.dp),
209             modifier = modifier
210         ) {
211             items(favorites) { post ->
212                 Card(
213                     modifier = Modifier.fillMaxWidth(),
214                     elevation =
215 CardDefaults.cardElevation(6.dp)
216                 ) {
217                     Row(modifier =
218 Modifier.padding(16.dp), verticalAlignment =
219 Alignment.CenterVertically) {
220                         post.imageResId?.let { resId ->
221                             Image(
222                                 painter =
223 painterResource(id = resId),
224                                 contentDescription =
225 "Gambar Doa",
226                                 modifier = Modifier
227                                     .size(80.dp)
228                                     .padding(end =
229 16.dp)
230                             )
231                         }
232                         Column(
233                             modifier =
234 Modifier.weight(1f)
235                         ) {
236                             Text(text = post.doa, style
237 = MaterialTheme.typography.titleMedium)
238                             Spacer(modifier =
239 Modifier.height(4.dp))
240                             Text(text = post.artinya,
241 style = MaterialTheme.typography.bodyMedium)
242                         }
243                         Button(
244                             onClick =
245 onRemoveFavorite(post) },

```

```

246                                     modifier                                     =
247 Modifier.height(36.dp)
248                                     ) {
249                                     Text("Hapus")
250                                     }
251                                 }
252                            }
253                        }
254                    }
255                }
256            }
257
258 @Composable
259 fun DoaItem(
260     post: Post,
261     navController: NavHostController,
262     onAddFavorite: (Post) -> Unit,
263 ) {
264     val context = LocalContext.current
265
266     Card(
267         modifier = Modifier.fillMaxWidth(),
268         elevation = CardDefaults.cardElevation(6.dp)
269     ) {
270         Row(modifier = Modifier.padding(16.dp)) {
271             post.imageResId?.let { resId ->
272                 Image(
273                     painter = painterResource(id =
274 resId),
275                     contentDescription = "Gambar Doa",
276                     modifier = Modifier
277                         .size(80.dp)
278                         .padding(end = 16.dp)
279                 )
280             }
281
282             Column(
283                 modifier = Modifier.weight(1f)
284             ) {
285                 Text(text = post.doa, style =
286 MaterialTheme.typography.titleMedium)
287                 Spacer(modifier =
288 Modifier.height(4.dp))
289
290                 Row(
291                     horizontalArrangement =
292 Arrangement.spacedBy(8.dp),

```

```

293         modifier = Modifier.fillMaxWidth()
294     ) {
295         Button(
296             onClick = {
297                 navController.navigate(
298
299 "detail/${post.doa}/${post.ayat}/${post.latin}/${post.
300 artinya}/${post.imageResId ?: 0}"
301             )
302         },
303         modifier = Modifier
304             .weight(1f)
305             .height(36.dp),
306         contentPadding =
307 PaddingValues(horizontal = 8.dp, vertical = 4.dp)
308     ) {
309         Text("Detail", style =
310 MaterialTheme.typography.bodyMedium)
311     }
312
313     Button(
314         onClick = {
315             val youtubeUrl =
316 "https://www.youtube.com/watch?v=G5O_TpkKibg"
317             val intent =
318 Intent(Intent.ACTION_VIEW, Uri.parse(youtubeUrl))
319
320 intent.setPackage("com.google.android.youtube")
321
322             try {
323
324 context.startActivity(intent)
325             } catch (e: Exception) {
326                 val browserIntent =
327 Intent(Intent.ACTION_VIEW, Uri.parse(youtubeUrl))
328
329 context.startActivity(browserIntent)
330             }
331         },
332         modifier = Modifier
333             .weight(1f)
334             .height(36.dp),
335         contentPadding =
336 PaddingValues(horizontal = 8.dp, vertical = 4.dp)
337     ) {
338         Text("App")
339     }

```


340	
341	Button(
342	onClick = { onAddFavorite(post)
343	},
344	modifier = Modifier
345	.weight(1f)
346	.height(36.dp),
347	contentPadding =
348	PaddingValues(horizontal = 8.dp, vertical = 4.dp)
349	) {
350	Text("Favorit")
351	}
352	}
353	}
354	}
355	}

ApiService.kt

Table 20. Source Code ApiService.kt

1	package com.example.modul5.data.api
2	
3	import com.example.modul5.data.models.Post
4	import retrofit2.http.GET
5	
6	interface ApiService {
7	@GET("api")
8	suspend fun getDoaList(): List<Post>
9	}

RetrofitInstance.kt

Table 21. Source Code RetrofitInstance.kt

1	package com.example.modul5.data.api
2	
3	import retrofit2.Retrofit
4	import retrofit2.converter.gson.GsonConverterFactory
5	
6	object RetrofitInstance {
7	private val retrofit by lazy {
8	Retrofit.Builder()
9	.baseUrl("https://doa-doa-api-
10	ahmadramadhan.fly.dev/")
11	
12	.addConverterFactory(GsonConverterFactory.create())

13	.build()
14	}
15	
16	val api: ApiService by lazy {
17	retrofit.create(ApiServices::class.java)
18	}
19	}

PostDao.kt

Table 22. Source Code PostDao.kt

1	package com.example.modul5.data.local
2	
3	import androidx.room.*
4	import com.example.modul5.data.models.Post
5	import kotlinx.coroutines.flow.Flow
6	
7	@Dao
8	interface PostDao {
9	@Query("SELECT * FROM post")
10	fun getAllPosts(): Flow<List<Post>>
11	
12	@Insert(onConflict = OnConflictStrategy.REPLACE)
13	suspend fun insertPosts(posts: List<Post>)
14	
15	@Query("DELETE FROM post")
16	suspend fun clearPosts()
17	}

PostDatabase.kt

Table 23. Source Code PostDatabase.kt

1	package com.example.modul5.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	import com.example.modul5.data.models.Post
8	
9	@Database(entities = [Post::class], version = 1)
10	abstract class PostDatabase : RoomDatabase() {
11	abstract fun postDao(): PostDao
12	
13	companion object {

14	@Volatile private var INSTANCE: PostDatabase? =
15	null
16	
17	fun getDatabase(context: Context): PostDatabase
18	{
19	return INSTANCE ?: synchronized(this) {
20	Room.databaseBuilder(
21	context.applicationContext,
22	PostDatabase::class.java,
23	"post_database"
24	).build().also { INSTANCE = it }
25	}
26	}
27	}
28	}

Post.kt

Table 24. Source Code Post.kt

1	package	com.example.modul5.data.models
2		
3	import	androidx.room.Entity
4	import	androidx.room.PrimaryKey
5		
6	@Entity(tableName	= "post")
7	data	class Post(
8	@PrimaryKey	val id: String,
9	val	doa: String,
10	val	ayat: String,
11	val	latin: String,
12	val	artinya: String,
13	val	imageResId: Int? = null
14	)	

PostRepository.kt

Table 25. Source Code PostRepository.kt

1	package	com.example.modul5.data.repository
2		
3	import	com.example.modul5.data.api.ApiServices
4	import	com.example.modul5.data.local.PostDao
5	import	com.example.modul5.data.models.Post
6	import	kotlinx.coroutines.flow.Flow
7		

8	class				PostRepository(
9	private	val	api:	ApiServices,	
10	private	val	dao:	PostDao	
11	)				{
12	fun	getPosts():	Flow<List<Post>>	=	dao.getAllPosts()
13					
14	suspend	fun	fetchAndCachePosts()		{
15	try				{
16		val	response	=	api.getDoaList()
17		dao.clearPosts()			
18		dao.insertPosts(response)			
19		}	catch	(e: Exception)	{
20		}			
21	}				
22	}				

DetailScreen.kt

Table 26. Source Code DetailScreen.kt

1	package		com.example.modul5.screens
2			
3	import		androidx.compose.foundation.Image
4	import		androidx.compose.foundation.layout.*
5	import		androidx.compose.material3.MaterialTheme
6	import		androidx.compose.material3.Text
7	import		androidx.compose.runtime.Composable
8	import		androidx.compose.ui.Modifier
9	import		androidx.compose.ui.res.painterResource
10	import		androidx.compose.ui.unit.dp
11	import		androidx.compose.ui.unit.sp
12			
13	@Composable		
14	fun	DetailScreen(doa: String, ayat: String, latin: String,	
15	artinya: String,	imageResId: Int)	{
16	Column(		
17	modifier	=	Modifier
18	.fillMaxSize()		
19	.padding(16.dp)		
20	)		{
21	Image(		
22	painter = painterResource(id = imageResId),		
23	contentDescription =	"Gambar Detail",	
24	modifier =	Modifier	
25	.fillMaxWidth()		
26	.height(200.dp)		
27	)		

28	Spacer(modifier	=	Modifier.height(16.dp))
29			
30	Text(text	=	"Judul:", style =
31	MaterialTheme.typography.titleMedium)		
32	Text(text = doa, fontSize = 20.sp, modifier =		
33	Modifier.padding(bottom	=	8.dp))
34			
35	Text(text	=	"Ayat:", style =
36	MaterialTheme.typography.titleMedium)		
37	Text(text = ayat, fontSize = 18.sp, modifier =		
38	Modifier.padding(bottom	=	8.dp))
39			
40	Text(text	=	"Latin:", style =
41	MaterialTheme.typography.titleMedium)		
42	Text(text = latin, fontSize = 18.sp, modifier =		
43	Modifier.padding(bottom	=	8.dp))
44			
45	Text(text	=	"Artinya:", style =
46	MaterialTheme.typography.titleMedium)		
47	Text(text = artinya, fontSize = 18.sp)		
48	}		
49	}		

PostViewModel.kt

Table 27. Source Code PostViewModel.kt

1	package	com.example.modul5.viewModel
2		
3	import	androidx.lifecycle.ViewModel
4	import	androidx.lifecycle.viewModelScope
5	import	com.example.modul5.R
6	import	com.example.modul5.data.models.Post
7	import	com.example.modul5.data.repository.PostRepository
8	import	kotlinx.coroutines.flow.*
9	import	kotlinx.coroutines.launch
10		
11	class	PostViewModel(private val repository:
12	PostRepository)	: ViewModel() {
13		
14	private	val _errorMessage =
15	MutableStateFlow<String?>(null)	
16	val errorMessage: StateFlow<String?>	= _errorMessage
17		
18	private	val _favorites =
19	MutableStateFlow<List<Post>>(emptyList())	
20	val favorites: StateFlow<List<Post>>	= _favorites

```

21
22     private      val      imageMap      =      mapOf (
23         "1"      to      R.drawable.satu,
24         "2"      to      R.drawable.dua,
25         "3"      to      R.drawable.tiga,
26         "4"      to      R.drawable.empat,
27         "5"      to      R.drawable.lima,
28         "6"      to      R.drawable.enam,
29         "7"      to      R.drawable.satu,
30         "8"      to      R.drawable.dua,
31         "9"      to      R.drawable.tiga,
32         "10"     to      R.drawable.empat,
33         "11"     to      R.drawable.lima,
34         "12"     to      R.drawable.enam,
35         "13"     to      R.drawable.satu,
36         "14"     to      R.drawable.dua,
37         "15"     to      R.drawable.tiga,
38         "16"     to      R.drawable.empat,
39         "17"     to      R.drawable.lima,
40         "18"     to      R.drawable.enam,
41         "19"     to      R.drawable.satu,
42         "20"     to      R.drawable.dua,
43         "21"     to      R.drawable.tiga,
44         "22"     to      R.drawable.empat,
45         "23"     to      R.drawable.lima,
46         "24"     to      R.drawable.enam,
47         "25"     to      R.drawable.satu,
48         "26"     to      R.drawable.dua,
49         "27"     to      R.drawable.tiga,
50         "28"     to      R.drawable.empat,
51         "29"     to      R.drawable.lima,
52         "30"     to      R.drawable.enam,
53         "31"     to      R.drawable.satu,
54         "32"     to      R.drawable.dua,
55         "33"     to      R.drawable.tiga,
56         "34"     to      R.drawable.empat,
57         "35"     to      R.drawable.lima,
58         "36"     to      R.drawable.enam,
59         "37"     to      R.drawable.satu
60
61     )
62
63     // Ambil data cached dari database, langsung
64     observable
65     val      posts:      StateFlow<List<Post>>      =
66     repository.getPosts ()
67         .map      {      list      ->

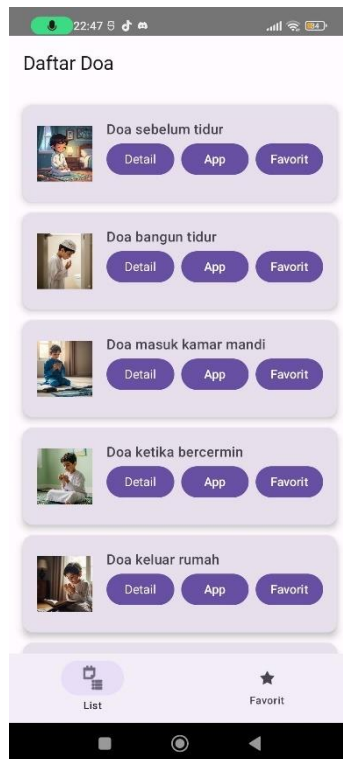
```

```

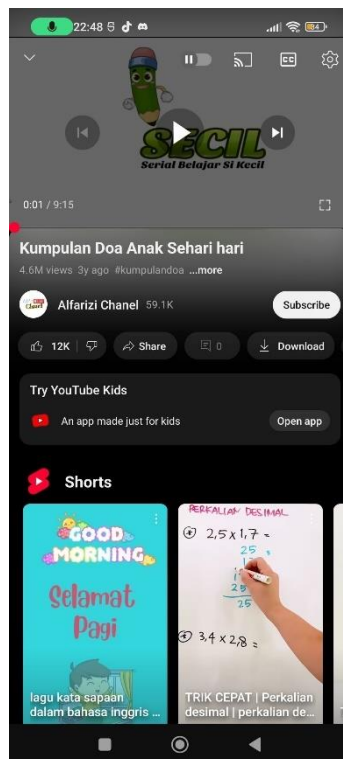
68         list.map { post ->
69             post.copy(
70                 imageResId = imageMap[post.id] ?:
71 R.drawable.placeholder
72             )
73         }
74     }
75     .stateIn(viewModelScope, SharingStarted.Lazily,
76 emptyList())
77
78     init {
79         refreshPosts()
80     }
81
82     private fun refreshPosts() {
83         viewModelScope.launch {
84             try {
85                 repository.fetchAndCachePosts()
86                 _errorMessage.value = null
87             } catch (e: Exception) {
88                 _errorMessage.value = "Error:
89 ${e.localizedMessage}"
90             }
91         }
92     }
93
94     fun addToFavorite(post: Post) {
95         if (!_favorites.value.contains(post)) {
96             _favorites.value = _favorites.value + post
97         }
98     }
99
100    fun removeFromFavorite(post: Post) {
101        _favorites.value = _favorites.value - post
102    }
103 }

```

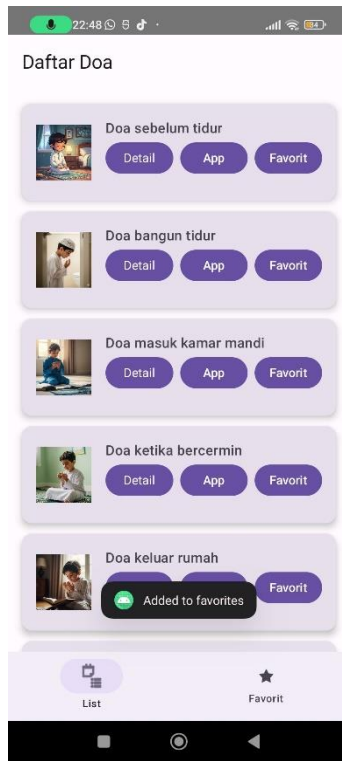
B. Output Program



Gambar 14. Screenshot Halaman Awal



Gambar 15. Screenshot Ketika Tombol Ditekan Mengarah Ke Aplikasi Youtube



Gambar 16. Screenshot Ketika Tombol Favorite Ditekan



Gambar 17. Screenshot Halaman Favorite



Gambar 18. Screenshot Ketika Tombol Hapus Dari Favorite Ditekan

C. Pembahasan

MainActivity.kt:

Pada line 1, dideklarasikan nama package file Kotlin yaitu com.example.modul5.

Pada line 3-16, diimport berbagai kelas dan fungsi dari Android dan Jetpack Compose untuk membangun UI, navigasi, lifecycle, serta akses data.

Pada line 18, dideklarasikan kelas MainActivity yang merupakan turunan dari ComponentActivity.

Pada line 20-31, didefinisikan kelas PostViewModelFactory sebagai pabrik pembuatan ViewModel yang menerima PostRepository sebagai parameter dan menghasilkan instance PostViewModel.

Pada line 33, anotasi @OptIn untuk menggunakan API experimental Material3 pada fungsi onCreate.

Pada line 34-36, override fungsi onCreate yang dipanggil saat activity dibuat.

Pada line 37, memanggil `super.onCreate` agar siklus hidup activity tetap berjalan normal.

Pada line 39-41, membuat instance API Retrofit, akses DAO database lokal, dan membuat repository data.

Pada line 43, membuat instance factory ViewModel menggunakan repository.

Pada line 45, memanggil `setContent` untuk menyiapkan UI Compose.

Pada line 46, membuat instance PostViewModel menggunakan factory yang sudah dibuat.

Pada line 48-50, mengumpulkan data posts, favoritePosts, dan errorMessage dari postViewModel menggunakan `collectAsState`.

Pada line 52, membuat navController untuk navigasi antar layar.

Pada line 53, mendeklarasikan variabel `selectedTab` dengan nilai awal "list" menggunakan `remember` dan `mutableStateOf`.

Pada line 55-77, membuat layout utama menggunakan Scaffold dengan TopAppBar dan NavigationBar sebagai bottomBar.

Pada line 79, mendefinisikan context yang mengambil konteks aplikasi saat ini.

Pada line 81-86, membuat lambda `onAddFavoriteWithToast` yang menambahkan post ke favorit dan menampilkan pesan toast.

Pada line 87-92, membuat lambda `onRemoveFavoriteWithToast` yang menghapus post dari favorit dan menampilkan pesan toast.

Pada line 94-115, menggunakan `when` untuk menentukan tampilan berdasarkan nilai `selectedTab`.

Pada line 95-103, jika tab "list" dipilih dan ada pesan error, tampilkan pesan error dengan warna tema error.

Pada line 104-111, jika tidak ada error, tampilkan daftar doa melalui composable `DoaList`.

Pada line 113-115, jika tab "favorit" dipilih, tampilkan layar favorit menggunakan composable `FavoritScreen`.

Pada line 119-136, definisi fungsi composable `DoaList` untuk menampilkan list doa menggunakan `LazyColumn`.

Pada line 137-164, definisi fungsi composable FavoritScreen untuk menampilkan list favorit atau pesan kosong jika daftar favorit kosong.

Pada line 166-212, definisi fungsi composable DoaItem untuk menampilkan satu item doa dengan tombol Detail, App (untuk membuka YouTube), dan Favorit.

ApiServices.kt

Pada line 1, dideklarasikan nama package file Kotlin yaitu com.example.modul5.data.api.

Pada line 3, diimport kelas Post dari package com.example.modul5.data.models yang merepresentasikan model data doa.

Pada line 4, diimport anotasi GET dari Retrofit yang digunakan untuk mendefinisikan endpoint HTTP GET.

Pada line 6, dideklarasikan interface ApiService sebagai kontrak untuk layanan API.

Pada line 7, mendefinisikan fungsi getDoaList dengan anotasi @GET("api") yang menandakan HTTP GET request ke endpoint "api".

Pada line 8, fungsi getDoaList dideklarasikan sebagai suspend agar bisa dipanggil secara coroutine, dan akan mengembalikan daftar objek Post (List<Post>).

RetrofitInstance.kt

Pada line 1, dideklarasikan nama package Kotlin yaitu com.example.modul5.data.api.

Pada line 3, diimport kelas Retrofit dari library Retrofit untuk melakukan HTTP client.

Pada line 4, diimport GsonConverterFactory untuk mengkonversi data JSON menjadi objek Kotlin.

Pada line 6, dideklarasikan objek RetrofitInstance sebagai singleton untuk mengelola instance Retrofit.

Pada line 7, properti retrofit dideklarasikan dengan by lazy supaya hanya dibuat sekali saat pertama kali dipanggil.

Pada line 8-12, membangun instance Retrofit menggunakan builder dengan:

line 9: mengatur base URL API "https://doa-doa-api-ahmadramadhan.fly.dev/".

line 10: menambahkan converter Gson untuk parsing JSON.

line 11: membangun instance Retrofit.

Pada line 14-17, properti api dideklarasikan dengan `by lazy` untuk membuat implementasi interface `ApiServices` menggunakan instance `Retrofit` yang sudah dibuat.

PostDao.kt

Pada line 1, dideklarasikan nama package Kotlin yaitu `com.example.modul5.data.local`.

Pada line 3, diimport semua komponen dari library Room (`@Dao`, `@Query`, `@Insert`, dll).

Pada line 4, diimport kelas `Post` sebagai model data yang digunakan dalam database.

Pada line 5, diimport `Flow` dari Kotlin Coroutines untuk mengelola data secara reaktif.

Pada line 7, dideklarasikan interface `PostDao` yang berisi fungsi akses data untuk tabel `post`.

Pada line 8-10, fungsi `getAllPosts()` dengan anotasi `@Query` yang mengambil semua data dari tabel `post` dan mengembalikan `Flow` dari list `Post` untuk observasi data secara real-time.

Pada line 12-14, fungsi `insertPosts()` dengan anotasi `@Insert` yang menyisipkan list `Post` ke database, jika ada konflik data akan mengganti (`REPLACE`).

Pada line 16-18, fungsi `clearPosts()` dengan anotasi `@Query` yang menghapus semua data dari tabel `post`.

PostDaoDatabase.kt

Pada line 1, dideklarasikan nama package Kotlin yaitu `com.example.modul5.data.local`.

Pada line 3-5, diimport kelas dan fungsi dari Android Room untuk membuat database lokal.

Pada line 6, diimport kelas model `Post` yang menjadi entitas database.

Pada line 8, anotasi `@Database` menandai kelas ini sebagai database Room dengan entitas `Post` dan versi database 1.

Pada line 9, deklarasi kelas abstrak `PostDatabase` yang mewarisi `RoomDatabase`.

Pada line 10, deklarasi fungsi abstrak `postDao()` yang akan mengembalikan DAO `PostDao` untuk mengakses tabel `post`.

Pada line 12-22, companion object sebagai singleton untuk mengelola instance database agar hanya dibuat satu kali selama aplikasi berjalan.

Pada line 13, anotasi `@Volatile` memastikan variabel `INSTANCE` terlihat konsisten di semua thread.

Pada line 14, deklarasi variabel `INSTANCE` yang menyimpan instance database `PostDatabase`.

Pada line 16-22, fungsi `getDatabase()` yang mengembalikan instance database, membuatnya jika belum ada dengan cara sinkronisasi thread agar aman dari kondisi race.

Pada line 18-21, pemanggilan `Room.databaseBuilder()` untuk membuat database dengan context aplikasi, kelas database `PostDatabase`, dan nama database `"post_database"`, lalu menginisialisasi `INSTANCE`.

Post.kt

Pada line 1, dideklarasikan nama package Kotlin yaitu `com.example.modul5.data.models`.

Pada line 3-4, diimport anotasi `Entity` dan `PrimaryKey` dari Room untuk menandai kelas sebagai entitas database dan menentukan primary key-nya.

Pada line 6, anotasi `@Entity` dengan parameter `tableName = "post"` menandai kelas ini sebagai entitas database dengan nama tabel `post`.

Pada line 7, deklarasi data class `Post` sebagai model data untuk menyimpan informasi doa.

Pada line 8, properti `id` bertipe `String` ditandai sebagai primary key dengan anotasi `@PrimaryKey`.

Pada line 9, properti `doa` bertipe `String` menyimpan teks doa.

Pada line 10, properti `ayat` bertipe `String` menyimpan ayat terkait doa.

Pada line 11, properti `latin` bertipe `String` menyimpan transliterasi latin dari doa.

Pada line 12, properti `artinya` bertipe `String` menyimpan arti doa dalam bahasa Indonesia.

Pada line 13, properti `imageResId` bertipe `Int?` (nullable) menyimpan ID resource gambar terkait doa, dengan nilai default `null` jika tidak ada gambar.

PostRepository.kt

Pada line 1, dideklarasikan package Kotlin `com.example.modul5.data.repository`.

Pada line 3-6, diimport class `ApiServices`, `PostDao`, `Post`, dan `Flow` untuk digunakan di dalam repository.

Pada line 8-12, deklarasi class `PostRepository` dengan dua properti private: `api` dari tipe `ApiServices` dan `dao` dari tipe `PostDao`, yang berfungsi sebagai sumber data remote dan lokal.

Pada line 13, fungsi `getPosts()` mengembalikan `Flow` daftar `Post` yang didapatkan dari database lokal melalui `dao.getAllPosts()`.

Pada line 15-22, fungsi `fetchAndCachePosts()` bertipe `suspend` yang akan memanggil API untuk mengambil data doa terbaru, lalu menghapus data lama di database lokal, dan memasukkan data baru tersebut ke database.

Pada line 17, pemanggilan API `api.getDoaList()` yang mengambil daftar doa secara remote.

Pada line 18, memanggil fungsi `dao.clearPosts()` untuk menghapus data doa lama dari database lokal.

Pada line 19, memanggil fungsi `dao.insertPosts(response)` untuk memasukkan data doa baru ke database lokal.

Pada line 20-21, blok `catch` menangkap dan mengabaikan kemungkinan exception jika terjadi kesalahan saat pemanggilan API atau penyimpanan data.

DetailScreen.kt

Pada line 1, dideklarasikan nama package file Kotlin yaitu `com.example.modul5.screen`.

Pada line 3, diimport `Image` dari `androidx.compose.foundation` untuk menampilkan gambar pada UI.

Pada line 4, diimport semua elemen layout dari `androidx.compose.foundation.layout` seperti `Column`, `Spacer`, dan modifier layout lainnya.

Pada line 5, diimport `MaterialTheme` dari `androidx.compose.material3` untuk menggunakan tema Material Design versi 3 pada komponen UI.

Pada line 6, diimport `Text` dari `androidx.compose.material3` untuk menampilkan teks pada layar.

Pada line 7, diimport anotasi `@Composable` dari `androidx.compose.runtime` yang menandakan fungsi ini sebagai fungsi composable di Jetpack Compose.

Pada line 8, diimport `Modifier` dari `androidx.compose.ui` yang digunakan untuk memodifikasi tampilan dan perilaku komponen UI.

Pada line 9, diimport `painterResource` dari `androidx.compose.ui.res` yang berfungsi untuk mengambil resource gambar berdasarkan ID.

Pada line 10, diimport `dp` (density-independent pixels) dari `androidx.compose.ui.unit` untuk mengatur ukuran dalam satuan dp.

Pada line 11, diimport `sp` (scale-independent pixels) dari `androidx.compose.ui.unit` untuk mengatur ukuran font secara responsif.

Pada line 13, dideklarasikan fungsi composable `DetailScreen` dengan parameter: `doa`, `ayat`, `latin`, artinya bertipe `String`, dan `imageResId` bertipe `Int` sebagai resource ID gambar.

Pada line 14, dibuat sebuah `Column` yang berfungsi menata isi UI secara vertikal.

Pada line 15, diterapkan modifier pada `Column` dengan `Modifier.fillMaxSize()` agar kolom mengisi seluruh ruang layar.

Pada line 16, modifier `padding(16.dp)` menambahkan jarak tepi sebesar 16 dp di sekeliling kolom.

Pada line 18, mulai blok konten `Column` yang berisi elemen-elemen UI.

Pada line 19, komponen `Image` ditampilkan dengan parameter:

`painter`: mengambil gambar dari resource dengan ID `imageResId`.

Pada line 20, diberikan deskripsi gambar dengan `contentDescription` bernilai "Gambar Detail" untuk aksesibilitas.

Pada line 21, modifier untuk gambar:

`fillMaxWidth()`: agar gambar melebar mengikuti lebar layar.

Pada line 22, modifier `height(200.dp)` memberi tinggi gambar sebesar 200 dp.

Pada line 24, Spacer digunakan untuk memberi jarak vertikal sebesar 16 dp antar elemen.

Pada line 26, Text menampilkan label "Judul:" dengan style titleMedium dari tema Material.

Pada line 27, Text menampilkan isi doa dengan ukuran font 20 sp dan padding bawah 8 dp sebagai jarak ke elemen berikutnya.

Pada line 29, Text menampilkan label "Ayat:" dengan style yang sama seperti sebelumnya.

Pada line 30, Text menampilkan isi ayat dengan ukuran font 18 sp dan padding bawah 8 dp.

Pada line 32, Text menampilkan label "Latin:" dengan style yang sama.

Pada line 33, Text menampilkan isi latin dengan font 18 sp dan padding bawah 8 dp.

Pada line 35, Text menampilkan label "Artinya:" dengan style yang sama.

Pada line 36, Text menampilkan isi artinya dengan ukuran font 18 sp.

PostViewModel.kt

Pada line 1, dideklarasikan nama package file Kotlin yaitu com.example.modul5.viewModel.

Pada line 3, diimport kelas ViewModel dari androidx.lifecycle sebagai dasar kelas ViewModel untuk mengelola UI-related data.

Pada line 4, diimport viewModelScope untuk mengatur coroutine scope yang terkait dengan siklus hidup ViewModel.

Pada line 5, diimport resource R dari package com.example.modul5 untuk akses resource gambar dan lainnya.

Pada line 6, diimport model data Post yang merepresentasikan data utama yang digunakan.

Pada line 7, diimport PostRepository sebagai sumber data dari API dan database lokal.

Pada line 8, diimport semua elemen flow dari kotlinx.coroutines.flow untuk pemrograman reaktif dan observable data stream.

Pada line 9, diimport fungsi launch dari coroutine untuk menjalankan tugas asynchronous.

Pada line 11, dideklarasikan kelas PostViewModel yang menerima PostRepository sebagai parameter konstruktor dan mewarisi ViewModel.

Pada line 13, mendefinisikan _errorMessage sebagai MutableStateFlow yang menyimpan pesan error bertipe nullable String, awalnya bernilai null.

Pada line 14, mendefinisikan errorMessage sebagai StateFlow yang hanya dapat dibaca, mengacu pada _errorMessage.

Pada line 16, mendefinisikan _favorites sebagai MutableStateFlow yang menyimpan daftar favorit Post, awalnya daftar kosong.

Pada line 17, mendefinisikan favorites sebagai StateFlow yang hanya dapat dibaca, mengacu pada _favorites.

Pada line 19-55, didefinisikan imageMap sebagai Map yang menghubungkan id Post (tipe String) dengan resource gambar drawable dari R.drawable. Map ini berfungsi untuk menetapkan gambar berdasarkan id post.

Pada line 58-66, mendefinisikan posts sebagai StateFlow<List<Post>> yang berasal dari repository.getPosts(), lalu:

map: memetakan setiap list post dengan menambahkan nilai imageResId berdasarkan imageMap. Jika id tidak ditemukan, menggunakan gambar placeholder.

stateIn: mengubah flow menjadi StateFlow dengan scope viewModelScope, strategi sharing SharingStarted.Lazily, dan nilai awal list kosong.

Pada line 68-70, blok init yang langsung memanggil fungsi refreshPosts() ketika ViewModel diinisialisasi.

Pada line 72-81, fungsi refreshPosts yang menjalankan coroutine di viewModelScope untuk: Memanggil fetchAndCachePosts() dari repository agar data terbaru diambil dan disimpan. Jika berhasil, errorMessage disetel ke null. Jika terjadi exception, errorMessage diisi dengan pesan error yang diterima.

Pada line 83-88, fungsi addToFavorite(post: Post) untuk menambah sebuah post ke daftar favorit jika belum ada di daftar.

Pada line 90-92, fungsi removeFromFavorite(post: Post) untuk menghapus sebuah post dari daftar favorit.

TAUTAN GIT

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/kkeshiian/Pemrograman-Mobilefix>